

Universidad Internacional de la Rioja (UNIR)

**Escuela Superior de Ingeniería y
Tecnología**

Master universitario en Inteligencia Artificial

Diseño de un asistente institucional agéntico: observabilidad, RAG y herramientas

Trabajo Fin de Máster

Presentado por: Nuria Iglesias Traviesa

Director: David Jiménez Cabello

Ciudad: Valencia

Fecha: 22/04/2026

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Planteamiento del trabajo	2
1.3. Estructura del trabajo	4
2. Contexto y estado del arte	6
2.1. Contexto	6
2.2. Estado del arte	7
2.2.1. Agentes y orquestación	8
2.2.2. Uso de herramientas e integración de capacidades	8
2.2.3. RAG	9
2.2.4. Evaluación y mejora concreta del retrieval	10
2.2.5. Asistentes institucionales	11
2.3. Encaje de la propuesta	12
2.4. Conclusiones	13
3. Objetivos y Metodología	15
3.1. Objetivos Generales	15
3.2. Objetivos Específicos	15
3.3. Criterios de comprobación de los objetivos	16
3.4. Metodología del trabajo	17
3.4.1. Scrum	18
3.4.2. Roles	18
3.4.3. Artefactos	18
3.4.4. Eventos	19
3.4.5. Adaptación al proyecto	19
3.5. Riesgos y planes de contingencia	19

3.6. Planificación del trabajo	21
3.6.1. Sprint 1: análisis y definición del problema	21
3.6.2. Sprint 2: diseño de la arquitectura y preparación del entorno de trabajo	21
3.6.3. Sprint 3: implementación del <i>baseline</i> del asistente	22
3.6.4. Sprint 4: integración de la funcionalidad accionable	22
3.6.5. Sprint 5: implementación de la mejora del componente de recuperación de información	22
3.6.6. Sprint 6: diseño de la evaluación y comparación experimental	23
3.6.7. Sprint 7: Revisión final, análisis de resultados y redacción de la memoria	23
3.7. Recursos y costes	24
3.7.1. Recursos humanos	24
3.7.2. Recursos técnicos	24
3.7.3. Estimación de dedicación	25
3.7.4. Estimación de costes	25
4. Diseño e implementación de la propuesta	26
4.1. Arquitectura general del sistema	26
4.2. Flujo de ejecución del asistente	27
4.3. Papel del orquestador	28
4.4. Herramientas disponibles	29
4.5. Variantes implementadas	29
4.6. Lenguaje y entorno de desarrollo	30
4.6.1. Python	30
4.6.2. Docker	31
4.7. Procesamiento documental	31
4.7.1. Obtención del corpus y scraping web	31
4.7.2. Normalización, segmentación y metadatos	33
4.8. Almacenamiento vectorial	34
4.9. Modelos de representación semántica	35
4.9.1. BGE-M3	35
4.10. Recuperación de información	36
4.10.1. Recuperación semántica	36
4.10.2. Recuperación híbrida con BM25	37
4.11. Mejora de la recuperación mediante reranking	37
4.11.1. Modelo de reranking seleccionado	39

4.12. Modelo de lenguaje	40
4.13. Interfaz de usuario y observabilidad	41
4.13.1. Streamlit	41
4.13.2. Phoenix	41
5. Resultados experimentales	43
5.1. Diseño de la evaluación	43
5.1.1. Corpus y dataset de evaluación	43
5.1.2. Métricas empleadas	45
5.1.3. Variantes evaluadas (ablation study)	46
5.2. Evaluación sobre dataset privado: UNIR	47
5.2.1. Resultados de recuperación documental	47
5.2.2. Resultados de calidad de respuesta	48
5.2.3. Análisis cualitativo de casos representativos	49
5.2.4. Evaluación funcional del orquestador	51
5.3. Evaluación sobre dataset público: XQuAD-es	52
5.3.1. Resultados de recuperación	52
5.3.2. Resultados de generación	53
5.3.3. Comparación con la evaluación interna	54
5.4. Discusión de resultados	54
6. Conclusiones y trabajo futuro	57
6.1. Resumen de la contribución	57
6.2. Valoración del cumplimiento de los objetivos	57
6.3. Principales resultados	60
6.4. Limitaciones	61
6.5. Trabajo futuro	62
6.6. Reflexión final	63
Bibliography	64
Appendices	69
Apéndice A. Recursos del proyecto	70

Resumen

Este Trabajo de Fin de Máster presenta el diseño, implementación y evaluación de un asistente institucional basado en *Retrieval-Augmented Generation* (RAG) e integración de herramientas. La arquitectura propuesta combina recuperación documental sobre un corpus de fuentes institucionales, mejora del *ranking* de evidencias mediante recuperación híbrida y *reranking*, orquestación del flujo de interacción basado en agente y generación y envío supervisado de correos electrónicos.

El sistema se evalúa mediante un *ablation study* con cuatro variantes progresivas sobre un dataset de 71 preguntas anotadas manualmente sobre documentación pública de UNIR, analizando el impacto de cada componente sobre métricas de recuperación (Hit@1, Hit@3, MRR) y de generación (Fact-Coverage, ROUGE-L, BERTScore).

Los resultados muestran que la recuperación híbrida y el *reranking* incrementan la cobertura factual un 35,5 % respecto al *baseline* (de 0.389 a 0.527) y mejoran la cobertura en el top-3 (Hit@3 = 0.918). El orquestador aporta valor en términos de control del flujo, abstención ante preguntas no respondibles y activación de herramientas accionables, con una calidad semántica equivalente a la variante determinista (BERTScore 0.648 frente a 0.656). En conjunto, los resultados sugieren la utilidad de una arquitectura modular, trazable y supervisada para asistentes institucionales privados.

Palabras clave: Modelos de lenguaje de gran tamaño, Retrieval-Augmented Generation, recuperación híbrida, reranking, orquestación, uso de herramientas, asistente institucional.

Abstract

This Master's Thesis presents the design, implementation and evaluation of an institutional assistant based on Retrieval-Augmented Generation (RAG) and tool integration. The proposed architecture combines document retrieval over an institutional corpus, evidence ranking improvement through hybrid retrieval and reranking, agent-based interaction flow orchestration, and supervised generation and sending of emails.

The system is evaluated through a four-variant ablation study on a manually annotated dataset of 71 questions over public UNIR documentation, measuring the impact of each component on retrieval metrics (Hit@1, Hit@3, MRR) and generation metrics (Fact-Coverage, ROUGE-L, BERTScore).

Results show that hybrid retrieval and reranking improve fact coverage by 35.5 % over the baseline (from 0.389 to 0.527) and increase top-3 coverage (Hit@3 = 0.918). The orchestrator contributes flow control, abstention on unanswerable questions and activation of actionable tools, with semantic quality equivalent to the deterministic variant (BERTScore 0.648 vs 0.656). Overall, the results suggest the usefulness of a modular, traceable and supervised architecture for private institutional assistants.

Keywords: Large Language Models, Retrieval-Augmented Generation, hybrid retrieval, reranking, orchestration, tool use, institutional assistant.

Índice de figuras

1.	Arquitectura general del asistente institucional propuesto.	4
2.	Diagrama de Gantt con la planificación temporal del trabajo.	23
3.	Pipeline de recuperación híbrida: BGE-M3 y BM25 se fusionan mediante RRF antes de ser reordenadas por el reranker.	39
4.	Interfaz conversacional del asistente implementada con Streamlit.	41

Índice de Tablas

1.	Estimación de horas dedicadas al proyecto.	25
2.	Estimación orientativa de costes humanos del proyecto.	25
3.	Resumen del proceso de construcción del corpus documental.	33
4.	Comparativa de bases de datos vectoriales consideradas.	35
5.	Resumen del corpus y del dataset utilizados en la evaluación.	44
6.	Distribución de preguntas del dataset de evaluación.	45
7.	Ejemplos representativos del dataset de evaluación con fuente y hechos espe- rados anotados.	45
8.	Diseño del <i>ablation study</i> : variantes evaluadas.	46
9.	Resultados de recuperación del <i>ablation study</i>	47
10.	Resultados del <i>ablation study</i> : calidad de respuesta sobre preguntas respondibles.	48
11.	Ejemplo cualitativo 1: pregunta de un documento con éxito parcial (variante C).	50
12.	Ejemplo cualitativo 2: elaboración del orquestador sin mejora de Fact-Coverage (variante C vs. D).	50
13.	Ejemplo cualitativo 3: abstención ante pregunta no respondible.	51
14.	Evaluación funcional del orquestador con métricas objetivas (n = número de pre- guntas de cada tipo).	51
15.	Resultados de recuperación sobre XQuAD-es (1.190 preguntas, 240 contextos únicos).	52
16.	Resultados de generación sobre XQuAD-es (corpus XQuAD temporal, 25 pre- guntas por variante).	53
17.	Comparación de métricas de recuperación entre la evaluación interna (UNIR) y XQuAD-es.	54
18.	Resumen de las métricas principales por variante.	60

Capítulo 1

Introducción

En este primer capítulo se presenta el sistema de agentes con integración de herramientas orientadas a la realización de acciones útiles para el usuario, así como la propuesta de mejora del componente de recuperación de información. Asimismo, se describen la motivación y el contexto del trabajo, el planteamiento general de la solución, los objetivos a grandes rasgos y la estructura del documento.

1.1. Motivación

La motivación principal de este trabajo surge de la necesidad de explorar cómo diseñar un asistente institucional que combine consulta documental y capacidad de realizar acciones. Frente a enfoques centrados exclusivamente en la generación de respuestas, se parte de la idea de que un asistente útil en un entorno institucional debe ser capaz no solo de informar, sino también de ayudar a materializar el siguiente paso de manera controlada y supervisada por el ser humano.

Esta necesidad resulta especialmente visible en el ámbito universitario, donde parte de la información relevante para el estudiante se encuentra dispersa en normativas, páginas web, documentos administrativos o guías internas cuya consulta no siempre es sencilla. Cuestiones relacionadas con trámites, prácticas, horarios o becas pueden generar dudas incluso cuando la información existe, simplemente porque no resulta fácilmente accesible o comprensible. En este contexto, disponer de un asistente capaz de responder de forma clara, fundamentada y contextualizada puede suponer una mejora significativa en la experiencia del usuario, especialmente si se tiene en cuenta que muchas instituciones universitarias todavía no cuentan con herramientas conversacionales suficientemente útiles, especializadas o integradas para este tipo de consultas.

A ello se añade una segunda motivación de carácter investigador. En sistemas basados en *Retrieval-Augmented Generation* (RAG), la calidad de la recuperación documental condiciona directamente la calidad de las respuestas generadas. Por este motivo, resulta de interés analizar si una mejora concreta del componente de recuperación, basada en estrategias de recuperación híbrida y *reranking* (Nogueira y col., 2020), puede traducirse en una mejora observable en las respuestas y en la utilidad general del asistente. De este modo, el trabajo no solo responde a una necesidad práctica de desarrollo, sino también a una pregunta aplicada de investigación sobre el impacto real de optimizar el *retrieval* en asistentes institucionales.

1.2. Planteamiento del trabajo

Se propone el desarrollo de un asistente conversacional institucional privado que permita consultar documentación interna, ofrecer respuestas fundamentadas y apoyar acciones útiles para el usuario dentro de un entorno controlado.

La contribución principal de este Trabajo de Fin de Máster no se limita al desarrollo de un chatbot convencional, sino que se centra en el diseño, implementación y evaluación de una arquitectura modular, trazable y supervisada para un asistente institucional. La propuesta combina de forma integrada cuatro capacidades complementarias: la recuperación documental sobre un corpus institucional, la mejora del ranking de evidencias mediante recuperación híbrida y *reranking*, la orquestación del flujo de interacción y la generación y envío supervisado de correos electrónicos.

Esta arquitectura responde a requisitos especialmente relevantes en entornos institucionales, como el control del flujo, la privacidad de la información, la trazabilidad de las respuestas y la supervisión humana de las acciones propuestas. De este modo, el asistente no se concibe únicamente como una interfaz conversacional para responder preguntas, sino como un sistema capaz de transformar una consulta documental en una respuesta fundamentada y, cuando procede, en una acción operativa supervisada. La aportación del trabajo reside, por tanto, en articular dentro de una misma solución conocimiento documental, mejora de la recuperación, decisión sobre el flujo de ejecución y apoyo accionable al usuario.

Para evaluar el rendimiento de la propuesta, se definirá un conjunto de datos de prueba compuesto por documentación institucional y un conjunto de consultas representativas del dominio. Estas consultas cubrirán distintos escenarios de uso, incluyendo preguntas informativas, consultas sobre procedimientos y peticiones orientadas a la generación de una acción posterior, como la redacción de un correo formal. Sobre este conjunto se comparará el comportamiento de un *baseline* basado en búsqueda semántica con una variante mejorada basada en búsque-

da híbrida y *reranking*, considerando tanto métricas de recuperación como la fundamentación de las respuestas generadas y la utilidad de las salidas producidas.

El asistente no se limita a responder preguntas, sino que puede facilitar tareas útiles para el usuario, como la generación y envío supervisado de correos electrónicos: el sistema propone el borrador, el usuario lo revisa y confirma, y solo entonces se realiza el envío. Por último, el trabajo incluye una vertiente experimental centrada en la mejora del componente de recuperación de información, mediante la comparación entre un *baseline* inicial y una estrategia mejorada basada en recuperación híbrida y *reranking*.

La Figura 1 muestra la arquitectura general del asistente propuesto. El sistema se organiza en dos grandes bloques principales. Por una parte, se distingue una fase offline de preparación del corpus, en la que las fuentes documentales se recopilan, transforman, segmentan e indexan para su posterior recuperación. Por otra parte, se define una fase online de ejecución, en la que el usuario interactúa con el asistente a través de una interfaz conversacional.

En esta fase online, el agente orquestador actúa como componente central de coordinación. Su función es interpretar la consulta del usuario, decidir qué capacidades deben activarse y coordinar el uso de las herramientas disponibles. Entre estas herramientas se incluye el componente de recuperación documental, encargado de consultar la base documental formada por Qdrant y BM25, así como una herramienta de correo electrónico con envío supervisado por el usuario. Finalmente, el modelo de lenguaje utiliza la información recuperada y el flujo decidido por el orquestador para generar la respuesta final o activar la herramienta de correo electrónico correspondiente.

Además, la arquitectura incorpora una capa de observabilidad basada en Phoenix y Open-Telemetry. Esta capa permite registrar trazas del comportamiento del sistema, revisar los documentos recuperados, analizar las llamadas al modelo y facilitar la depuración del pipeline durante las fases de desarrollo y evaluación.

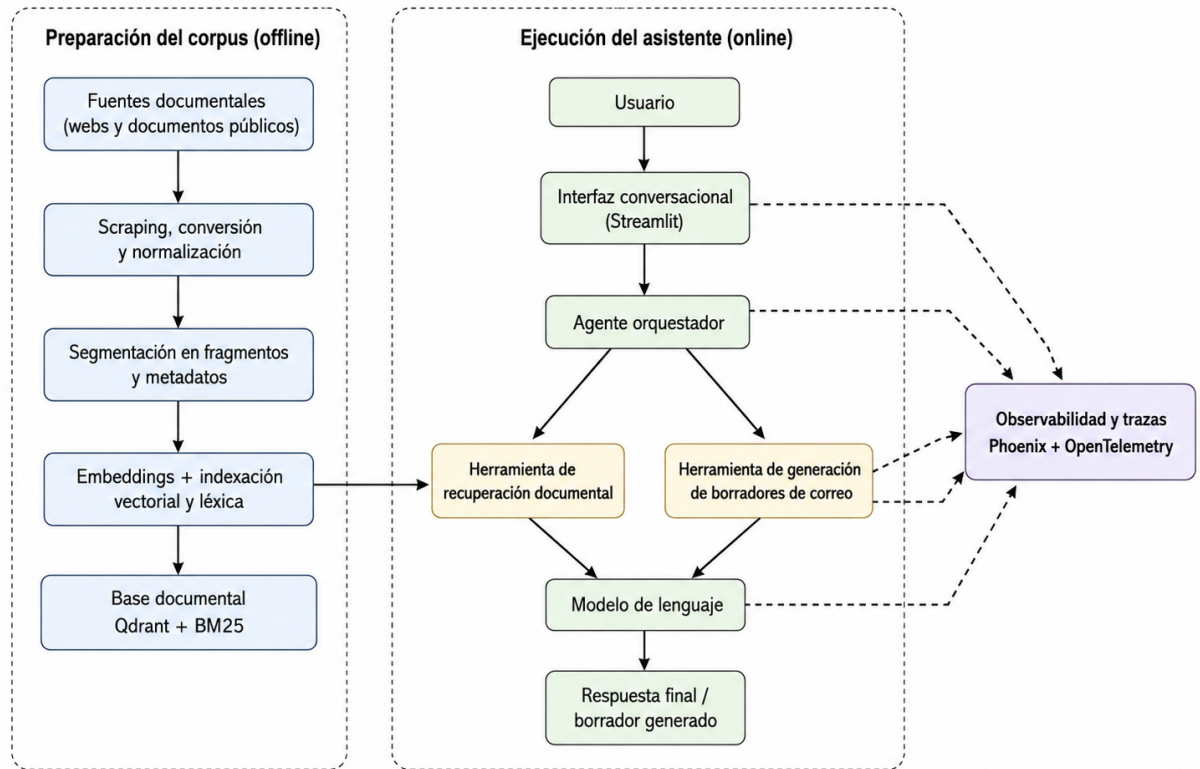


Figura 1: Arquitectura general del asistente institucional propuesto.

1.3. Estructura del trabajo

La memoria se organiza en varios capítulos que permiten abordar de forma progresiva tanto la definición del problema y la planificación del trabajo como su fundamentación teórica, su desarrollo técnico y su validación experimental.

De forma más concreta, la estructura del documento es la siguiente:

- **Capítulo 1. Introducción.** Presenta el contexto y la motivación del trabajo, el planteamiento general de la solución y la organización de la memoria.
- **Capítulo 2. Contexto y estado del arte.** Revisa los antecedentes más relevantes sobre agentes basados en modelos de lenguaje, orquestación, uso de herramientas, sistemas de recuperación aumentada, mejora del *retrieval* y asistentes institucionales especializados.
- **Capítulo 3. Objetivos y metodología.** Define los objetivos generales y específicos del trabajo, los criterios de comprobación planteados, la metodología seguida, la planificación, los riesgos identificados y los recursos necesarios.

- **Capítulo 4. Diseño e implementación de la propuesta.** Describe la arquitectura general del asistente institucional, el papel del agente orquestador, el sistema de recuperación documental, la integración de herramientas accionables y las principales decisiones técnicas adoptadas durante el desarrollo del prototipo.
- **Capítulo 5. Evaluación experimental y discusión de resultados.** Presenta el conjunto de evaluación, el *baseline* de recuperación, la variante mejorada basada en búsqueda híbrida y *reranking*, las métricas empleadas, los resultados obtenidos y su análisis en relación con los objetivos planteados y las limitaciones del sistema.
- **Capítulo 6. Conclusiones y trabajo futuro.** Resume las principales aportaciones del trabajo, valora el grado de cumplimiento de los objetivos planteados y expone posibles líneas de mejora o ampliación futura.

Capítulo 2

Contexto y estado del arte

En este capítulo se describe el contexto del trabajo y se revisa el estado del arte de las tecnologías y enfoques más relevantes para la propuesta. Además, se analiza el encaje de la solución planteada dentro de la literatura revisada y se exponen las principales conclusiones obtenidas.

2.1. Contexto

En los últimos años, el desarrollo de la inteligencia artificial generativa ha impulsado la aparición de agentes conversacionales cada vez más capaces de desenvolverse en escenarios reales. Una de las líneas más relevantes en esta evolución ha sido la incorporación de mecanismos de acceso a conocimiento externo, de modo que los modelos pueden consultar documentación, bases de datos y servicios externos para fundamentar mejor sus respuestas. En este contexto, los enfoques basados en *Retrieval-Augmented Generation* (RAG) (Lewis y col., 2020) han adquirido un papel central, al permitir combinar las capacidades generativas de los modelos con la recuperación de documentación relevante.

Al mismo tiempo, los modelos de lenguaje han dejado de concebirse únicamente como interfaces conversacionales y han pasado a integrarse en sistemas capaces de apoyar flujos de trabajo más amplios. Esta evolución resulta especialmente significativa en entornos institucionales, donde las consultas de los usuarios no siempre finalizan en una simple respuesta informativa, sino que suelen desembocar en una acción posterior, como la redacción de un correo o la realización de un trámite. En este tipo de contextos aparecen además exigencias adicionales, ya que el sistema debe operar sobre documentación interna, respetar restricciones de acceso a la información y ofrecer un mayor grado de control sobre su comportamiento.

2.2. Estado del arte

La literatura reciente ha explorado con especial interés las arquitecturas de agentes y los mecanismos de orquestación. En esta línea, ReAct (Yao y col., 2023) plantea una combinación entre razonamiento y acción, mientras que trabajos como AgentVerse (Chen y col., 2024b), MetaGPT (Hong y col., 2024) y AutoGen (Wu y col., 2024) avanzan hacia esquemas de colaboración y coordinación entre agentes. Más recientemente, Dang et al. (Dang y col., 2025) abordan la orquestación multiagente como un mecanismo dinámico de coordinación. De forma complementaria, otra línea de trabajo relevante se centra en el uso de herramientas y APIs externas por parte de modelos de lenguaje. Toolformer (Schick y col., 2023) introduce la capacidad de aprender cuándo y cómo invocar herramientas, mientras que Gorilla (Patil y col., 2024) y ToolLLM (Qin y col., 2024) exploran la conexión de modelos de lenguaje con APIs reales. En esta misma dirección, API-Bank (Li y col., 2023) propone un benchmark específico para evaluar sistemas aumentados con herramientas. Asimismo, los sistemas de Retrieval-Augmented Generation fueron introducidos por Lewis et al. (Lewis y col., 2020) como un enfoque para combinar modelos generativos con recuperación de conocimiento externo. Trabajos posteriores como REPLUG (Shi y col., 2024), Self-RAG (Asai y col., 2024) y Adaptive-RAG (Jeong y col., 2024) han profundizado en distintas formas de integrar, controlar o adaptar el proceso de recuperación. Finalmente, también se han propuesto asistentes especializados para dominios institucionales, universitarios o cerrados. Marcel (Trientes y col., 2025) se orienta al soporte conversacional de estudiantes universitarios, URASys (Nguyen y col., 2025) aborda consultas ambiguas o no respondibles en contextos académicos, y HyPA-RAG (Kalra y col., 2025) aplica recuperación híbrida y adaptación de parámetros en un dominio especializado.

Estas líneas resultan especialmente relevantes porque la propuesta no se limita a diseñar un sistema conversacional genérico, sino un asistente institucional con capacidad para consultar documentación interna, utilizar herramientas accionables y transformar una respuesta fundamentada en una actuación útil para el usuario. Además, el trabajo incorpora una contribución experimental específica centrada en la mejora del componente de recuperación de información mediante una estrategia de recuperación híbrida y *reranking*. Por ello, este estado del arte no solo revisa los fundamentos técnicos del sistema, sino también sitúa la propuesta en relación con soluciones institucionales existentes y con las principales líneas de mejora del retrieval.

2.2.1. Agentes y orquestación

Una de las líneas más influyentes en este ámbito es la que entiende al agente basado en modelos de lenguaje no como un simple generador de texto, sino como una entidad capaz de razonar, planificar y actuar sobre el entorno. En esta dirección, Yao et al. (Yao y col., 2023) proponen ReAct, un enfoque que combina trazas de razonamiento con acciones explícitas ejecutadas sobre fuentes externas o entornos interactivos. Para el contexto de este trabajo, ReAct resulta especialmente relevante porque introduce una base conceptual clara para el diseño de un agente orquestador que no solo responde, sino que decide qué hacer, qué consultar y en qué orden ejecutar cada paso.

Sobre esta idea inicial, la investigación posterior ha ampliado el foco desde agentes individuales hacia sistemas compuestos por múltiples agentes especializados, como AgentVerse (Chen y col., 2024b), un marco de colaboración multiagente en el que distintos agentes cooperan para resolver tareas. Los resultados descritos por los autores muestran que, en determinados escenarios, un conjunto coordinado de agentes puede superar a un único agente generalista.

Hong et al. (Hong y col., 2024) proponen MetaGPT, que, frente a aproximaciones más informales, introduce una organización más estructurada del trabajo entre agentes, de forma que cada componente participa en una fase concreta del proceso y verifica resultados intermedios. Sugiere que la división explícita de responsabilidades puede reducir inconsistencias y facilitar la incorporación de nuevas capacidades sin rediseñar completamente el sistema.

Por su parte, Wu et al. (Wu y col., 2024) presentan AutoGen. Uno de los elementos más destacables es que los agentes pueden operar en distintos modos, combinando modelos de lenguaje, intervención humana y uso de herramientas externas. Esta visión encaja con el enfoque de este proyecto, en el que se persigue una arquitectura en la que un agente orquestador coordine módulos y capacidades desacopladas dentro de un entorno controlado.

Más recientemente, Dang et al. (Dang y col., 2025) plantean un esquema centralizado en el que un orquestador dirige de forma adaptativa a los distintos agentes según el estado de la tarea. Aportan evidencias de que la orquestación debe entenderse como un mecanismo de coordinación capaz de reorganizar prioridades, seleccionar agentes adecuados y ajustar el flujo de ejecución en función del contexto.

2.2.2. Uso de herramientas e integración de capacidades

Una de las limitaciones más relevantes de los modelos de lenguaje es su dificultad para interactuar de forma fiable con recursos externos. Por este motivo, una línea de investigación

cada vez más importante se centra en el *tool use*, entendido como la capacidad de un modelo para decidir cuándo utilizar una herramienta y cómo incorporar el resultado obtenido a la respuesta final.

Uno de los trabajos de referencia en esta línea es Toolformer (Schick y col., 2023). Su contribución principal es demostrar que el modelo no solo puede ejecutar llamadas a APIs, sino también aprender a decidir en qué momento conviene hacerlo y qué argumentos debe pasar. Esta propuesta aporta una base conceptual sólida para entender la invocación de herramientas como una capacidad integrada dentro del comportamiento del agente.

Una contribución complementaria es ToolLLM, presentada por Qin et al. (Qin y col., 2024). En este caso, los autores proponen entrenar y evaluar modelos de lenguaje en escenarios de uso de herramientas sobre un conjunto masivo de APIs reales. Para el contexto de este trabajo, ToolLLM refuerza la idea de que una arquitectura modular debe diseñarse pensando en la extensibilidad, evitando acoplamientos fuertes entre el agente y cada herramienta concreta.

Junto a estas propuestas, también resulta necesario considerar trabajos centrados en su evaluación. En este sentido, Li et al. (Li y col., 2023) presentan API-Bank, un *benchmark* en el que se evalúan aspectos como la planificación, la recuperación de la API correcta y la ejecución de llamadas.

2.2.3. RAG

La técnica de *Retrieval-Augmented Generation* (RAG) surge como una respuesta a una de las limitaciones fundamentales de los modelos de lenguaje: su dependencia exclusiva del conocimiento almacenado en los parámetros del modelo. Aunque este conocimiento permite obtener un buen rendimiento en múltiples tareas, presenta problemas importantes cuando se requiere acceder a información específica o perteneciente a un dominio concreto.

En este contexto, Lewis et al. (Lewis y col., 2020) introducen RAG como un marco que combina un modelo generativo con una memoria externa, accesible mediante un mecanismo de recuperación. Su relevancia radica en que permite paliar las limitaciones de los modelos de lenguaje en dominios específicos al proporcionarles evidencia externa actualizada. No obstante, el uso de estos sistemas plantea retos significativos en el componente de recuperación: la búsqueda semántica simple a menudo presenta dificultades para capturar términos técnicos precisos, gestionar la ambigüedad de las consultas o garantizar que los documentos recuperados sean los más relevantes para la generación final. Afrontar estos retos justifica la necesidad de evolucionar el modelo hacia un componente de recuperación de información más robusto, que actúe como base sólida para la consulta y fundamentación de las respuestas.

La investigación posterior ha explorado distintas variantes orientadas a hacer el uso de RAG más flexible y aplicable en sistemas reales. En esta línea, Shi et al. (Shi y col., 2024) proponen REPLUG, donde tratan al modelo de lenguaje como una caja negra y utilizan un recuperador ajustable cuyos documentos se añaden directamente al *prompt* del modelo. Demostraron que los beneficios de RAG no dependen necesariamente de modificar la arquitectura del modelo, sino que pueden alcanzarse mediante mecanismos externos de recuperación e integración de contexto.

Asai et al. (Asai y col., 2024) presentan Self-RAG, un enfoque en el que el modelo aprende no solo a recuperar información, sino también a reflexionar sobre la necesidad de dicha recuperación y a criticar la calidad de la respuesta generada. Esta contribución desplaza de la idea de un RAG estático hacia uno más adaptativo y autorreflexivo.

Jeong et al. (Jeong y col., 2024) proponen Adaptive-RAG, una arquitectura que selecciona dinámicamente la estrategia de recuperación más adecuada en función de la complejidad de la pregunta. De este modo, la literatura reciente sugiere que la calidad de un sistema RAG depende de la capacidad de decidir cómo y cuándo recuperar evidencia.

Una línea de investigación especialmente importante es la relacionada con la privacidad en sistemas RAG. En este sentido, Zeng et al. (Zeng y col., 2024) estudian cómo un sistema RAG puede exponer contenido de la base de recuperación y evidencian que el uso de información privada no solo aporta beneficios en términos de utilidad, sino que también exige nuevas medidas de protección. Refuerza el considerar la privacidad no como una característica accesorio, sino como una restricción de diseño de primer nivel.

2.2.4. Evaluación y mejora concreta del retrieval

Esta cuestión resulta especialmente relevante, ya que una de sus contribuciones previstas consiste precisamente en proponer y evaluar una mejora concreta sobre el componente de recuperación frente a un *baseline* inicial.

Un punto de partida fundamental en esta línea es el trabajo de Karpukhin et al. (Karpukhin y col., 2020) sobre *Dense Passage Retrieval* (DPR). DPR plantea aprender representaciones vectoriales en las que cada pregunta quede cerca de los documentos más relevantes, de manera que sea posible recuperarlos de forma eficiente incluso en grandes colecciones de documentos.

A partir de esta base, varios trabajos han abordado la optimización del entrenamiento de recuperadores densos. Entre ellos, Qu et al. (Qu y col., 2021) proponen RocketQA, que introduce mejoras específicas como la selección de negativos poco informativos o la existencia de

falsos negativos en el entrenamiento. Esta aportación demuestra que una mejora del *retrieval* no tiene por qué implicar rediseñar todo el pipeline, sino que puede consistir en una forma específica el entrenamiento o la selección de evidencia recuperada.

Nogueira et al. (Nogueira y col., 2020) demuestran que un modelo generativo puede emplearse eficazmente para calcular relevancia documental y reordenar documentos previamente recuperados. Esta estrategia resulta atractiva porque permite aumentar la calidad del contexto recuperado sin comprometer excesivamente la arquitectura.

No obstante, mejorar el *retrieval* no es suficiente si no se dispone de criterios adecuados para medir su impacto. En este punto cobra especial relevancia el trabajo de Saad-Falcon et al. (Saad-Falcon y col., 2024) sobre ARES, el cual propone evaluar separadamente distintas dimensiones del sistema, como la relevancia del contexto recuperado y la relevancia global de la respuesta generada. En lugar de limitar la evaluación a métricas, refuerza la idea de que conviene analizar también la calidad del contexto recuperado.

2.2.5. Asistentes institucionales

Debido al contexto de este trabajo, interesa analizar propuestas que hayan abordado asistentes en contextos institucionales y trabajos que hayan explorado el uso de herramientas en lenguaje natural.

En el ámbito universitario, Trienes et al. (Trienes y col., 2025) presentan Marcel, un agente conversacional ligero y de código abierto orientado al soporte de estudiantes, su alcance permanece centrado en la respuesta a consultas de admisión. La solución combina un módulo de *Frequently Asked Questions* con un esquema de *Retrieval-Augmented Generation* para fundamentar las respuestas con información verificable.

Una propuesta más robusta en escenarios cerrados es URASys, de Nguyen et al. (Nguyen y col., 2025), que plantea un sistema unificado para responder preguntas estándar y ambiguas en tareas de *academic advising*. Es especialmente relevante porque muestra que, en contextos institucionales, no basta con recuperar información, sino que también es necesario manejar ambigüedad y reducir alucinaciones. No obstante, no aborda de forma central la integración de herramientas.

Más allá del entorno universitario, encontramos varias propuestas. Liu et al. (Liu y col., 2025) proponen WorkTeam, un marco multiagente para transformar instrucciones en lenguaje natural en flujos de trabajo estructurados en entornos empresariales. Su arquitectura se compone de tres agentes especializados: un supervisor que planifica, un orquestador que organiza componentes, y un agente *filler* que completa parámetros a partir de documentación relevante. Este

trabajo transforma una instrucción textual a una acción compuesta sobre herramientas.

En una línea cercana al uso de herramientas, Zhang et al. (Zhang y col., 2025) presentan AskToAct. La idea central del trabajo consiste en utilizar los propios parámetros de las herramientas como representación explícita de la intención del usuario, generando automáticamente datos de entrenamiento a partir de consultas incompletas. Esta aportación refleja que, para ejecutar correctamente una herramienta, el sistema debe ser capaz de detectar qué información falta y solicitarla de manera adecuada.

Por último, Kalra et al. (Kalra y col., 2025) presentan HyPA-RAG, un sistema de *Retrieval-Augmented Generation* adaptativo para aplicaciones legales. La solución incorpora un clasificador de complejidad de la consulta para ajustar parámetros dinámicamente, una recuperación híbrida que combina métodos basados en grafos de conocimiento, y un marco de evaluación adaptado al dominio. Este trabajo es especialmente relevante para la parte investigadora, ya que demuestra que una mejora explícita del módulo de recuperación puede traducirse en ganancias en exactitud de recuperación, fidelidad y precisión contextual.

2.3. Encaje de la propuesta

La propuesta de este TFM se sitúa en la intersección entre asistentes institucionales especializados, sistemas de uso de herramientas y mejora experimental del componente de recuperación de información. Por una parte, se plantea un asistente institucional especializado orientado a resolver consultas sobre documentación interna. Por otra, incorpora una funcionalidad accionable: la generación y envío supervisado de correos electrónicos como paso operativo posterior a la consulta, donde el sistema presenta siempre un borrador al usuario y realiza el envío a través de la API de Microsoft Graph únicamente tras su confirmación explícita. Finalmente incorpora una contribución experimental específica sobre el componente de recuperación de información basada en recuperación híbrida y *reranking*.

La propuesta busca cerrar el ciclo entre consulta y acción, transformando una necesidad informativa en una respuesta fundamentada y, cuando procede, en una actuación útil, trazable y supervisada. Esta es precisamente la aportación diferencial del trabajo: combinar en una misma arquitectura capacidades que en la literatura suelen abordarse por separado, como la recuperación documental, la mejora del ranking, la orquestación del flujo y el uso controlado de herramientas.

Comparada con Marcel, nuestra propuesta comparte el interés por un asistente universitario basado en RAG, pero se diferencia en dos aspectos fundamentales. En primer lugar, el caso de uso no se restringe a admisión ni a orientación externa, sino que se orienta a soporte

institucional sobre documentación y procedimientos internos. En segundo lugar, el sistema no se limita a responder preguntas, sino que incorpora una transición explícita desde la evidencia documental hacia una acción operativa, en este caso la redacción supervisada de correos formales.

En relación con URASys, la propuesta coincide en la importancia de operar en entornos cerrados, reducir alucinaciones y tratar consultas difíciles. No obstante, mientras URASys concentra su aportación en la robustez del *question answering* frente a ambigüedad y no respondibles, este proyecto amplía el alcance hacia una arquitectura con capacidad de acción.

WorkTeam transforma instrucciones en *workflows* generales de negocio mediante una arquitectura multiagente de orquestación y relleno de parámetros. Esta propuesta, en cambio, parte de un escenario institucional centrado en la consulta documental y utiliza la acción como extensión natural de una respuesta fundamentada por RAG.

Finalmente, HyPA-RAG constituye el antecedente más cercano en la parte investigadora, ya que también propone mejorar el componente de recuperación de información mediante recuperación híbrida y ajuste adaptativo. No obstante, la diferencia principal es el contexto ya que HyPA-RAG se presenta como una solución especializada para consultas legales y de política pública.

Por tanto, la aportación diferencial radica en la combinación de elementos que, en los trabajos revisados, aparecen habitualmente por separado.

2.4. Conclusiones

La revisión realizada permite extraer varias conclusiones relevantes. En primer lugar, las aportaciones respaldan el diseño de una arquitectura modular con un agente orquestador que coordine herramientas y capacidades especializadas, especialmente cuando se busca extensibilidad, trazabilidad y control en entornos privados.

En segundo lugar, integrar herramientas de forma efectiva exige resolver cuestiones fundamentales: decidir cuándo es necesario recurrir a una capacidad externa, seleccionar la herramienta adecuada, construir correctamente la llamada y utilizar el resultado obtenido de manera coherente dentro del proceso de generación.

En tercer lugar, los trabajos revisados sobre *Retrieval-Augmented Generation* muestran que no siempre basta con recuperar información relevante, sino que también es necesario hacerlo mediante una arquitectura controlada y alineada con los requisitos de privacidad.

Asimismo, la literatura permite identificar varias estrategias principales para mejorar el componente de recuperación. Destacan los enfoques en varias etapas, donde una recuperación

inicial eficiente se complementa con un *reranker* más preciso. La investigación reciente subraya que la evaluación del *retrieval* debe abordarse de forma explícita, considerando no solo cuántos documentos relevantes se recuperan, sino también cómo afecta dicha recuperación a la fidelidad y utilidad de la respuesta final. Por ello, este bloque del estado del arte proporciona una base directa para la parte experimental, justificando la comparación entre un *baseline* de recuperación semántica y una mejora concreta basada en recuperación híbrida y *reranking*.

En conjunto, los trabajos revisados muestran la existencia de asistentes especializados en dominios institucionales o cerrados, el interés creciente por la coordinación de herramientas, y la mejora del propio componente de recuperación de información como vía activa de investigación aplicada. Sin embargo, estas líneas suelen aparecer separadas y este vacío justifica el interés de una propuesta que combine consulta documental fundamentada, uso de herramientas orientadas a la acción y mejora experimental del componente de recuperación de información dentro de un mismo asistente institucional.

Capítulo 3

Objetivos y Metodología

En este capítulo, se presentarán los objetivos generales y específicos del trabajo, junto con los criterios definidos para comprobar su cumplimiento. Posteriormente, se describirá la metodología adoptada para el desarrollo del proyecto, así como la planificación prevista, los riesgos identificados y los recursos necesarios para su realización.

3.1. Objetivos Generales

El objetivo general es desarrollar y evaluar un asistente institucional, capaz de consultar documentación interna mediante un sistema *Retrieval-Augmented Generation* (RAG) y de apoyar acciones operativas mediante herramientas integradas, con el fin de mejorar su utilidad en un entorno controlado.

De forma más específica, el trabajo busca validar una arquitectura en la que estas capacidades no se traten como módulos aislados, sino como partes coordinadas de un mismo flujo: recuperación de evidencia, mejora del ranking, generación de respuesta, decisión orquestada y acción supervisada.

A mayores se persigue analizar en qué medida una mejora del componente de recuperación, basada en recuperación híbrida y *reranking*, produce un efecto observable sobre la calidad de la evidencia recuperada y de las respuestas generadas, en comparación con un *baseline* de recuperación semántica simple.

3.2. Objetivos Específicos

Para alcanzar el objetivo general, se plantean los siguientes objetivos específicos:

1. **Identificar** los requisitos funcionales y no funcionales de un asistente institucional privado, atendiendo a aspectos como consulta documental, fundamentación de respuestas, trazabilidad, control del flujo y uso supervisado de herramientas.
2. **Diseñar** una arquitectura modular basada en un agente orquestador que coordine la recuperación de información, la generación de respuestas y la invocación de herramientas externas.
3. **Desarrollar** un prototipo funcional capaz de procesar consultas en lenguaje natural sobre documentación interna y generar respuestas apoyadas en evidencia recuperada.
4. **Integrar** una funcionalidad accionable basada en herramientas, centrada en la generación de correos electrónicos formales a partir de la información obtenida durante la consulta.
5. **Establecer** un *baseline* de recuperación basado en búsqueda semántica simple para disponer de una referencia inicial de funcionamiento.
6. **Implementar** una variante mejorada del componente de recuperación de información mediante una estrategia de recuperación híbrida y *reranking*.
7. **Definir** un conjunto de casos de prueba representativos del dominio institucional, incluyendo consultas informativas, consultas sobre procedimientos y consultas orientadas a acción.
8. **Comparar** el rendimiento del *baseline* y de la variante mejorada mediante métricas de recuperación, fundamentación de respuestas, utilidad de la salida generada y coste temporal del sistema.
9. **Evaluar** a través de métricas en qué medida la mejora introducida en el componente de recuperación repercute en la calidad global del asistente y en su capacidad para ofrecer respuestas útiles y operativamente aprovechables.
10. **Analizar** las fortalezas, limitaciones y posibles líneas de mejora de la solución propuesta en el contexto de asistentes institucionales privados y uso de herramientas.

3.3. Criterios de comprobación de los objetivos

Con el fin de que los objetivos anteriores puedan verificarse de forma objetiva, se emplearán los siguientes criterios de comprobación:

- El conjunto de evaluación deberá contener al menos 25 casos de prueba representativos del dominio institucional, incluyendo preguntas respondibles y preguntas no respondibles a partir del corpus disponible.
- El corpus documental deberá estar formado por documentos públicos del dominio institucional, procesados y segmentados en fragmentos adecuados para su recuperación posterior.
- La comparación entre el *baseline* y la variante mejorada se realizará mediante métricas de recuperación como Hit@1, Hit@3 y MRR.
- Se considerará que la mejora del componente de recuperación produce un efecto observable si la variante mejorada obtiene, al menos, una mejora de 3 puntos porcentuales en Hit@1 o Hit@3, o una mejora relativa del 5 % en MRR respecto al *baseline*. En este contexto, Hit@1 indica si la fuente esperada aparece en la primera posición, Hit@3 indica si aparece entre los tres primeros resultados y MRR mide la posición media recíproca de la primera fuente relevante recuperada.
- La calidad de las respuestas generadas se analizará mediante la cobertura de hechos esperados en las preguntas respondibles, comparando el comportamiento del flujo determinista y del orquestador.
- En las preguntas no respondibles, se evaluará la capacidad del sistema para abstenerse de responder cuando no exista evidencia suficiente en el corpus documental.
- En los casos orientados a acción, se evaluará la capacidad del orquestador para detectar la intención de envío de correo y activar correctamente la herramienta de correo electrónico. La herramienta genera siempre un borrador que el usuario revisa; el envío real a través de la API de Microsoft Graph se realiza únicamente tras la confirmación explícita del usuario.
- La evaluación funcional del orquestador comprobará si el sistema selecciona el flujo adecuado en consultas simples, consultas multi-documento, preguntas comparativas, preguntas no respondibles y solicitudes de correo electrónico.

3.4. Metodología del trabajo

En el desarrollo de este se adoptará una metodología ágil basada en Scrum, al considerarse un enfoque adecuado para organizar un proyecto tecnológico con una parte aplicada y otra

parte evaluativa. Esta elección se justifica por su flexibilidad, su carácter iterativo y su capacidad para adaptarse a cambios en los requisitos, en las decisiones de diseño o en los resultados obtenidos durante la implementación y la evaluación del sistema.

En este caso, el trabajo no consiste únicamente en desarrollar un prototipo funcional, sino también en analizar comparativamente el comportamiento de distintas configuraciones del componente de recuperación de información. Por ello, Scrum se empleará como marco de organización y seguimiento del desarrollo, permitiendo dividir el trabajo en incrementos manejables, revisar periódicamente el progreso y ajustar decisiones conforme avance el proyecto.

3.4.1. Scrum

Scrum es un marco de trabajo ágil orientado al desarrollo incremental de productos. Su funcionamiento se basa en iteraciones cortas y planificadas, denominadas *sprints*, al final de las cuales se obtiene un incremento funcional del sistema. Este enfoque resulta apropiado, ya que permite avanzar de forma progresiva desde la definición del problema hasta la implementación del prototipo y su evaluación experimental.

3.4.2. Roles

Dado el contexto académico del trabajo, los roles de Scrum se adaptarán de forma simplificada:

- **Product Owner:** asumido por la dirección, al encargarse de supervisar el enfoque general del trabajo, validar su orientación y priorizar los elementos más relevantes del proyecto.
- **Scrum Master:** también asumido de forma parcial por la dirección, en la medida en que orienta el desarrollo, ayuda a resolver bloqueos y supervisa la coherencia metodológica del trabajo.
- **Equipo de desarrollo:** asumido por la estudiante, responsable del análisis, diseño, implementación, experimentación y redacción de la memoria.

3.4.3. Artefactos

Los principales artefactos de trabajo serán los siguientes:

- **Product Backlog:** listado priorizado de tareas y objetivos del proyecto, incluyendo decisiones de arquitectura, preparación documental, implementación de componentes, diseño de experimentos y redacción.

- **Sprint Backlog:** subconjunto de tareas seleccionadas para cada sprint.
- **Incremento:** resultado funcional o evaluable obtenido al final de cada sprint, como puede ser una primera arquitectura definida, un *baseline* operativo o una versión mejorada del sistema.

3.4.4. Eventos

El trabajo se organizará mediante los eventos habituales de Scrum, adaptados a un proyecto individual:

- **Sprint Planning:** definición de las tareas a abordar en cada iteración.
- **Seguimiento:** revisión periódica del progreso, dificultades encontradas y ajustes necesarios.
- **Sprint Review:** revisión del resultado obtenido al final de cada sprint.
- **Sprint Retrospective:** valoración de los aspectos que han funcionado correctamente y de los elementos a mejorar en la siguiente iteración.

3.4.5. Adaptación al proyecto

Aunque Scrum se concibe habitualmente para equipos de desarrollo, su estructura puede adaptarse a un trabajo individual. Los *sprints* no se orientarán únicamente a implementar funcionalidades, sino también a producir resultados parciales verificables.

De este modo, la metodología de desarrollo permitirá combinar dos dimensiones del trabajo: por una parte, la construcción iterativa del asistente institucional privado; por otra, la comparación entre un *baseline* de recuperación semántica y una variante mejorada basada en recuperación híbrida y *reranking*.

3.5. Riesgos y planes de contingencia

Durante el desarrollo resulta necesario identificar posibles riesgos técnicos, metodológicos y de planificación que puedan afectar al alcance, a la calidad de la solución o al calendario previsto. Dado que el proyecto combina implementación de un prototipo funcional con una evaluación experimental comparativa, la gestión de riesgos adquiere especial relevancia para asegurar la viabilidad del trabajo.

A continuación, se describen los principales riesgos identificados y las medidas de contingencia previstas para reducir su impacto:

1. **Dificultades en la integración de componentes heterogéneos.** La integración entre el modelo de lenguaje, el componente de recuperación, la base documental y la funcionalidad accionable puede presentar incompatibilidades técnicas o requerir más esfuerzo de implementación del previsto. Como plan de contingencia, se priorizará una arquitectura modular y desacoplada, permitiendo validar cada componente por separado antes de su integración completa.
2. **Calidad insuficiente de la base documental o del conjunto de evaluación.** El rendimiento del asistente depende en gran medida de la calidad del corpus documental y de la representatividad de los casos de prueba. Un conjunto poco adecuado podría dificultar tanto el funcionamiento del sistema como la evaluación de la mejora propuesta. Como medida de contingencia, se realizará una selección y revisión temprana del corpus y de las consultas de prueba, garantizando que cubran los escenarios principales del dominio.
3. **Resultados experimentales poco concluyentes.** Existe la posibilidad de que la mejora basada en recuperación híbrida y *reranking* no produzca diferencias suficientemente claras respecto al *baseline*, o que dichas diferencias no sean tan amplias como se esperaba inicialmente. Como plan de contingencia, el trabajo no se orientará a demostrar una mejora absoluta, sino a realizar una comparación controlada y rigurosa entre ambas configuraciones.
4. **Incremento excesivo de la latencia del sistema.** La incorporación de una segunda etapa de recuperación o de un mecanismo de *reranking* puede aumentar el tiempo de respuesta del asistente y comprometer su utilidad práctica como prototipo. Para mitigar este riesgo, se medirá la latencia desde las primeras versiones funcionales y se limitará el número de documentos candidatos o el coste de los modelos auxiliares en caso necesario.
5. **Cambios en el alcance o en las decisiones técnicas durante el desarrollo.** Como sucede en muchos proyectos de desarrollo e investigación aplicada, algunas decisiones inicialmente previstas pueden necesitar ajustes conforme avance la implementación o la evaluación. Como medida de contingencia, se adoptará una planificación iterativa basada en Scrum, de forma que cada sprint permita revisar el estado del trabajo, reordenar prioridades y acotar el alcance sin perder coherencia con los objetivos definidos.

6. **Retrasos en el calendario previsto.** La simultaneidad entre implementación, experimentación y redacción de la memoria puede provocar desviaciones temporales respecto a la planificación inicial. Para reducir este riesgo, se organizará el trabajo en incrementos parciales y se reservará una fase final específica para revisión, análisis de resultados y redacción.
7. **Limitaciones derivadas del carácter individual del proyecto.** Al tratarse de un TFM desarrollado de forma individual, tanto el análisis como la implementación, la evaluación y la documentación recaen sobre una única persona, lo que puede ralentizar el avance en fases de mayor carga técnica. Como contingencia, se mantendrá una planificación realista, con objetivos acotados y verificables, evitando incorporar funcionalidades adicionales que no resulten imprescindibles para demostrar la contribución principal del trabajo.

3.6. Planificación del trabajo

De manera general, cada sprint tendrá una duración aproximada de entre dos y tres semanas, como se muestra en la Figura 2, aunque esta distribución podrá adaptarse en función de la complejidad real de las tareas y de las incidencias surgidas durante el desarrollo. Al final de cada iteración se realizará una revisión del avance conseguido y una replanificación de las tareas pendientes, con el fin de mantener el proyecto alineado con los objetivos establecidos.

3.6.1. Sprint 1: análisis y definición del problema

En esta primera fase se concretará el alcance del trabajo, se revisará el estado del arte ya recopilado y se terminarán de definir los requisitos funcionales y no funcionales del asistente institucional privado. Asimismo, se delimitará el caso de uso principal, se identificarán las capacidades mínimas del prototipo y se establecerán los criterios de evaluación que se utilizarán en la parte experimental.

Durante esta fase también se seleccionarán las herramientas y tecnologías necesarias para implementar el sistema, así como el enfoque general de la arquitectura.

3.6.2. Sprint 2: diseño de la arquitectura y preparación del entorno de trabajo

El segundo sprint estará centrado en el diseño técnico del sistema. En esta etapa se definirá la arquitectura modular del asistente, identificando los componentes principales, sus responsabilidades y el flujo de interacción entre ellos. En particular, se detallará el papel del agente

orquestador, el componente de recuperación documental, el modelo generativo y la funcionalidad de herramienta integrada.

Además, se preparará el entorno de desarrollo, se seleccionará o estructurará el corpus documental inicial y se organizará el material necesario para la construcción posterior del prototipo.

3.6.3. Sprint 3: implementación del *baseline* del asistente

En este sprint se desarrollará una primera versión funcional del sistema, que actuará como *baseline*. Esta configuración inicial permitirá procesar consultas en lenguaje natural, recuperar documentos relevantes mediante búsqueda semántica simple y generar respuestas apoyadas en la evidencia recuperada.

El objetivo de esta fase será disponer de una referencia operativa sobre la que poder comparar posteriormente la mejora investigadora del trabajo. Al finalizar este sprint deberá existir una primera versión ejecutable del asistente, aunque todavía limitada a su funcionalidad esencial.

3.6.4. Sprint 4: integración de la funcionalidad accionable

Una vez disponible el *baseline*, el siguiente sprint se centrará en ampliar el sistema con una capacidad accionable. En concreto, se integrará la generación y envío supervisado de correos electrónicos formales: el sistema genera un borrador a partir de la consulta del usuario y de la información recuperada, lo presenta para su revisión y realiza el envío únicamente tras la confirmación explícita del usuario.

Esta fase permitirá que el asistente no solo responda preguntas, sino que también apoye una actuación posterior útil dentro del entorno institucional. Al finalizar este sprint, el prototipo deberá ser capaz de producir una salida operativa básica en los casos orientados a acción.

3.6.5. Sprint 5: implementación de la mejora del componente de recuperación de información

El quinto sprint estará orientado a la principal aportación experimental del TFM. En esta etapa se implementará una variante mejorada del componente de recuperación de información basada en recuperación híbrida y *reranking*, manteniendo la posibilidad de compararla con el *baseline* definido anteriormente. El objetivo será introducir una mejora concreta y controlada

sobre el proceso de recuperación documental, sin modificar de forma radical el resto del sistema.

3.6.6. Sprint 6: diseño de la evaluación y comparación experimental

En este sprint se preparará y ejecutará la evaluación comparativa entre el *baseline* y la variante mejorada. Para ello, se terminará de construir el conjunto de casos de prueba, se definirán las rúbricas de evaluación manual y se recogerán las métricas seleccionadas para el análisis.

La comparación se centrará en dimensiones como la calidad de la recuperación, la fundamentación de las respuestas, la eficacia de la herramienta de correo electrónico y el coste temporal del sistema. Esta fase permitirá disponer de resultados cuantitativos con los que analizar el impacto real de la mejora propuesta.

3.6.7. Sprint 7: Revisión final, análisis de resultados y redacción de la memoria

La última fase del trabajo se dedicará a consolidar los resultados obtenidos, interpretar los datos recogidos durante la evaluación y redactar la memoria final del TFM. También se revisará el grado de cumplimiento de los objetivos planteados y se identificarán las principales limitaciones del sistema, así como posibles líneas de mejora o ampliación futura. Esta etapa incluirá además la revisión formal de la memoria.



Figura 2: Diagrama de Gantt con la planificación temporal del trabajo.

3.7. Recursos y costes

En esta sección se describen los recursos necesarios y se realiza una estimación orientativa de sus costes. Dado que se trata de un proyecto académico desarrollado de forma individual, la mayor parte del esfuerzo recae en la estudiante, mientras que la dirección del TFM asume funciones de supervisión, revisión y seguimiento.

3.7.1. Recursos humanos

Los recursos humanos considerados para el desarrollo del proyecto son los siguientes:

- **Dirección del TFM:** responsable de orientar el trabajo, revisar su evolución, validar las decisiones principales y supervisar la coherencia metodológica y académica del proyecto.
- **Estudiante:** responsable del análisis del problema, diseño de la arquitectura, implementación del prototipo, preparación del conjunto de evaluación, ejecución de los experimentos, análisis de resultados y redacción de la memoria final.

Aunque el proyecto es realizado por una única persona, desde el punto de vista de la estimación puede considerarse que la estudiante asume varias funciones de forma simultánea, entre ellas las de analista, desarrolladora e investigadora aplicada.

3.7.2. Recursos técnicos

Para el desarrollo del TFM se requerirán los siguientes recursos técnicos:

- Ordenador personal para desarrollo, pruebas y redacción de la memoria.
- Entorno de programación y herramientas de desarrollo para la implementación del prototipo.
- Herramientas de edición y documentación para la elaboración de diagramas, esquemas y memoria.
- Servicios o librerías necesarios para la implementación del sistema RAG, la integración del modelo generativo y el procesamiento documental.
- Herramientas de evaluación y análisis de resultados.

En términos generales, se priorizará el uso de herramientas gratuitas, académicas o ya disponibles en el entorno de trabajo, por lo que no se prevé un coste material elevado asociado

al proyecto. En caso de utilizar servicios externos de pago, estos quedarán limitados a un uso puntual o experimental.

3.7.3. Estimación de dedicación

La siguiente tabla recoge una estimación orientativa de las horas de trabajo necesarias para desarrollar el proyecto:

Rol	Horas proyecto	Horas memoria	Total horas
Estudiante	300	120	420
Dirección del TFM	20	10	30

Tabla 1: Estimación de horas dedicadas al proyecto.

La dedicación principal corresponde a la estudiante, ya que asume tanto el desarrollo técnico del prototipo como la parte experimental y documental del trabajo. En cambio, la dirección del TFM participa de manera periódica mediante reuniones de seguimiento, revisión de avances y supervisión de la memoria.

3.7.4. Estimación de costes

A efectos de estimación, se asigna un coste por hora orientativo a cada uno de los roles considerados. Estos valores no deben interpretarse como costes contractuales reales, sino como una aproximación académica al esfuerzo invertido en el proyecto.

Rol	Total horas	Coste/hora (€)	Coste total (€)
Estudiante	420	25	10.500
Dirección del TFM	30	35	1.050
Total	450	–	11.550

Tabla 2: Estimación orientativa de costes humanos del proyecto.

Por tanto, el coste total estimado del proyecto, considerando únicamente los recursos humanos, sería de 11.550€. Esta cifra representa una valoración aproximada del esfuerzo necesario para desarrollar el asistente, implementar la mejora experimental propuesta y documentar adecuadamente el trabajo realizado.

Capítulo 4

Diseño e implementación de la propuesta

En este capítulo se describe el diseño técnico del asistente institucional propuesto, así como las principales decisiones adoptadas durante su implementación. Para ello, se presenta la arquitectura general del sistema, el flujo de ejecución del asistente, el papel del agente orquestador y las herramientas disponibles. Finalmente, se exponen las variantes implementadas y las tecnologías utilizadas para el desarrollo del prototipo.

4.1. Arquitectura general del sistema

La arquitectura general del sistema se organiza en torno a dos fases principales: una fase offline de preparación del corpus y una fase online de ejecución del asistente. Esta separación permite distinguir claramente entre las tareas necesarias para construir la base documental y las tareas que se realizan en tiempo de consulta.

La fase offline comprende la obtención, normalización, segmentación e indexación de los documentos que forman parte del corpus institucional. En esta etapa, las fuentes documentales se recopilan a partir de páginas y documentos públicos, se transforman a un formato homogéneo, se dividen en fragmentos o *chunks* y se enriquecen con metadatos. Posteriormente, dichos fragmentos se representan mediante embeddings y se almacenan en una base documental preparada para su recuperación posterior.

La fase online comienza cuando el usuario introduce una consulta en la interfaz conversacional. A partir de esa entrada, el agente orquestador interpreta la petición y decide qué capacidades del sistema deben activarse. En consultas informativas, el flujo principal consiste en recuperar evidencia documental relevante y generar una respuesta fundamentada. En

cambio, cuando la petición tiene una intención accionable, el orquestador puede activar una herramienta específica, como la redacción y envío supervisado de un correo electrónico.

La Figura 1 muestra una visión global de esta arquitectura. En ella se observa cómo la base documental alimenta la herramienta de recuperación, cómo el orquestador coordina las capacidades disponibles y cómo la capa de observabilidad permite registrar trazas del comportamiento del sistema durante la ejecución.

4.2. Flujo de ejecución del asistente

El flujo de ejecución del asistente parte de una consulta formulada en lenguaje natural por el usuario. La entrada se recibe a través de la interfaz conversacional y se envía al núcleo del sistema, donde el orquestador determina el tipo de tarea que debe resolverse. Esta decisión condiciona el resto del proceso, ya que no todas las consultas requieren el mismo tratamiento.

- **Preguntas informativas simples.** El sistema sigue un flujo RAG directo: la consulta se transforma en una representación adecuada para el recuperador, se buscan los fragmentos documentales más relevantes dentro de la base documental y el modelo de lenguaje genera una respuesta utilizando como contexto la evidencia recuperada.
- **Preguntas que combinan varias fuentes.** El sistema puede descomponer la pregunta en subconsultas o recuperar un conjunto más amplio de fragmentos candidatos. Este comportamiento resulta útil en preguntas comparativas o en consultas que no pueden resolverse a partir de un único documento, donde el objetivo no es recuperar un fragmento aislado sino reunir evidencia suficiente para elaborar una respuesta más completa y estructurada.
- **Preguntas no respondibles.** Si la evidencia recuperada no resulta suficiente o no guarda relación clara con la consulta, el flujo favorece la abstención o una respuesta prudente que indique la falta de información disponible en el corpus. Este comportamiento es especialmente importante en un asistente institucional, donde proporcionar información incorrecta podría afectar negativamente a la confianza del usuario.
- **Consultas con intención accionable.** Cuando la consulta expresa una intención accionable, como solicitar ayuda para redactar un correo formal, el orquestador activa la herramienta de correo electrónico. El sistema genera el borrador a partir de la consulta y el contexto recuperado, lo presenta al usuario para su revisión y, solo tras la confirmación

explícita del usuario, realiza el envío a través de la API de Microsoft Graph. En ningún caso se produce el envío sin intervención humana previa.

4.3. Papel del orquestador

El agente orquestador actúa como componente de coordinación dentro del sistema. Su función principal consiste en interpretar la consulta del usuario, decidir qué flujo de ejecución resulta más adecuado y activar las herramientas necesarias para resolver la tarea. De este modo, el orquestador no sustituye al componente de recuperación ni al modelo de lenguaje, sino que organiza su uso dentro de una arquitectura controlada.

El orquestador implementado sigue el paradigma ReAct (Yao y col., 2023) (*Reasoning + Acting*), propuesto por Yao et al. como alternativa a las arquitecturas que separan razonamiento y acción en fases independientes. La idea central de ReAct es intercalar, dentro de un mismo bucle iterativo, trazas de razonamiento explícito con invocaciones a herramientas externas: el modelo no decide de antemano cuántas acciones ejecutará ni en qué orden, sino que infiere el siguiente paso a partir del resultado de la acción anterior, a diferencia de los *pipelines* deterministas, en los que la secuencia de operaciones está fijada en tiempo de diseño y no puede adaptarse a la respuesta del entorno.

En cada iteración del bucle, el LLM produce un bloque estructurado con un *Pensamiento*, la herramienta seleccionada (*Herramienta*) y su argumento (*Entrada*). El sistema ejecuta la herramienta y devuelve la observación al modelo, que la incorpora en el siguiente paso. Este ciclo *Pensamiento* → *Acción* → *Observación* se repite hasta que el modelo considera que dispone de evidencia suficiente para generar la respuesta o decide que la pregunta no es respondible, lo que permite al orquestador adaptar su comportamiento a cada consulta en lugar de seguir una secuencia fija predefinida.

El conjunto de herramientas disponibles delimita el ámbito de actuación del asistente.

1. *buscar*: recupera fragmentos relevantes del corpus para una consulta concreta; puede invocarse varias veces con distintas reformulaciones.
2. *rechazar*: señala que no existe evidencia suficiente para responder con rigor.
3. *redactar_correo*: genera un borrador de correo formal usando el contexto recuperado.
4. *responder*: indica que el LLM considera que ya dispone de evidencia suficiente y delega la generación de la respuesta final al componente RAG base.

Cualquier acción fuera de este conjunto es rechazada, convirtiendo al orquestador en una primera capa de *guardrails* del sistema.

4.4. Herramientas disponibles

Una herramienta se entiende como un componente externo al modelo de lenguaje que proporciona una capacidad concreta. En este contexto, el sistema incorpora herramientas especializadas que pueden ser utilizadas por el orquestador durante la ejecución.

La herramienta principal es la recuperación documental, que permite consultar la base documental construida previamente y devolver los fragmentos más relevantes para la pregunta del usuario. Su papel es fundamental en el sistema, ya que proporciona la evidencia necesaria para que las respuestas estén fundamentadas en información del corpus y no dependan únicamente del conocimiento paramétrico del modelo de lenguaje.

La segunda herramienta es la generación y envío supervisado de correos electrónicos. Esta funcionalidad permite transformar una consulta del usuario o una respuesta informativa previa en un texto formal orientado a una actuación posterior. El sistema genera el borrador a partir del contexto recuperado y lo presenta al usuario para su revisión; en ningún caso se realiza el envío de forma automática. Solo tras la confirmación explícita del usuario, el mensaje se envía a través de la API de Microsoft Graph con autenticación OAuth 2.0, manteniendo en todo momento la intervención humana como requisito previo a cualquier acción real.

El uso de herramientas se plantea de forma supervisada. El asistente puede proponer una acción, gestionar el envío supervisado de un correo o estructurar una respuesta accionable, pero la ejecución de estas herramientas queda bajo control del usuario. Este enfoque resulta adecuado para un entorno institucional, donde es importante mantener trazabilidad, evitar acciones no deseadas y conservar la intervención humana en decisiones sensibles.

4.5. Variantes implementadas

Con el fin de evaluar el impacto de los distintos componentes del sistema, se implementaron varias variantes. Estas permiten comparar una configuración inicial sencilla con versiones que incorporan mejoras específicas en la recuperación documental o en el control del flujo mediante orquestación.

La primera variante corresponde al *baseline* determinista. Su principal objetivo es servir como punto de referencia para analizar el efecto de las mejoras posteriores. Esta configuración sigue un flujo RAG directo: recibe la consulta del usuario, recupera fragmentos mediante

búsqueda semántica y genera una respuesta a partir del contexto obtenido.

La segunda variante introduce una mejora en el componente de recuperación de información. En este caso, la búsqueda semántica se complementa con recuperación léxica mediante BM25 y con una etapa posterior de *reranking*. El objetivo de esta variante es comprobar si una recuperación más robusta permite situar la evidencia relevante en posiciones más altas del ranking y, por tanto, mejorar la calidad del contexto proporcionado al modelo de lenguaje.

La tercera variante incorpora el agente orquestador basado en el paradigma ReAct. En esta configuración, el LLM actúa como motor de decisión en un bucle iterativo: razona sobre la evidencia recuperada, decide qué herramienta invocar a continuación y continúa hasta tener suficiente información para responder o determinar que no existe evidencia en el corpus. Esta variante permite analizar el valor del orquestador en tareas como la abstención ante preguntas no respondibles, la identificación de solicitudes accionables, la reformulación iterativa de consultas y la activación de herramientas especializadas.

Estas variantes se implementan de forma configurable, permitiendo activar o desactivar las componentes para facilitar la comparación experimental posterior, ya que cada configuración puede ejecutarse sobre el mismo corpus y el mismo conjunto de consultas de evaluación.

4.6. Lenguaje y entorno de desarrollo

4.6.1. Python

El lenguaje principal del proyecto es Python. Esta elección se debe a su amplia adopción en el ámbito de la inteligencia artificial, el procesamiento del lenguaje natural y la computación científica. Su ecosistema incluye bibliotecas consolidadas para el tratamiento de datos, el aprendizaje automático y el uso de modelos de lenguaje, como NumPy, SciPy, scikit-learn o Transformers (Harris y col., 2020; Virtanen y col., 2020; Pedregosa y col., 2011; Wolf y col., 2020). Esta disponibilidad de herramientas resulta especialmente adecuada para un prototipo basado en RAG, donde es necesario integrar procesamiento documental, generación de embeddings, recuperación semántica, reranking, evaluación y conexión con modelos generativos.

Por otro lado, nos encontramos con Java y Go, que son alternativas robustas para servicios backend con altos requisitos de concurrencia, mantenibilidad o rendimiento. Sin embargo, en este trabajo el objetivo principal no es optimizar una infraestructura de producción, sino diseñar, implementar y evaluar distintas variantes de una arquitectura RAG.

También se ha valorado el uso de lenguajes de menor nivel, como C++, que podrían aportar ventajas en términos de rendimiento. No obstante, gran parte del coste computacional del siste-

ma recae en la inferencia de modelos, la generación de embeddings y las consultas al almacén vectorial, componentes que ya suelen estar implementados mediante bibliotecas optimizadas o servicios externos. Por tanto, utilizar C++ para la lógica de orquestación o experimentación aumentaría la complejidad del desarrollo sin aportar una mejora proporcional al objetivo del prototipo.

Un último aspecto relevante es la reproducibilidad. Python permite aislar dependencias mediante entornos virtuales, de forma que cada entorno puede mantener sus propios paquetes y versiones sin interferir con la instalación global del sistema (Meyer, 2011). Esta característica facilita la ejecución controlada del prototipo, la comparación entre variantes experimentales y la futura replicación del trabajo.

4.6.2. Docker

Docker se utiliza como tecnología de contenedorización para ejecutar de forma aislada los servicios auxiliares del sistema. En este caso, su uso permite levantar componentes como Qdrant, utilizado como base de datos vectorial, y Phoenix, empleado para la observabilidad del pipeline, sin instalarlos directamente sobre el sistema operativo anfitrión.

Para la definición y ejecución de estos servicios se emplea Docker Compose (Docker Inc., 2026a), con un único fichero de configuración que define los servicios, redes y volúmenes necesarios para una aplicación multicontenedor. El uso de volúmenes permite conservar información entre ejecuciones, evitando que datos como las colecciones vectoriales o las trazas generadas se pierdan al detener o recrear los contenedores (Docker Inc., 2026b). Es extremadamente útil en un prototipo RAG, donde es necesario mantener el estado de servicios persistentes durante las fases de ingesta, evaluación y experimentación.

4.7. Procesamiento documental

El procesamiento documental constituye una fase esencial ya que afecta directamente a la calidad de la información que posteriormente será recuperada por el modelo. En este proyecto, dicha fase incluye la obtención de documentos, su conversión a un formato homogéneo y su segmentación en fragmentos adecuados para la recuperación semántica.

4.7.1. Obtención del corpus y scraping web

El corpus del proyecto se construye exclusivamente a partir de información disponible en URLs públicas del dominio institucional. Para ello, se desarrolló un proceso de scraping propio

basado en *httplib* y *BeautifulSoup*. La primera librería se utilizó para realizar peticiones HTTP de forma controlada, mientras que la segunda permitió analizar el contenido HTML, extraer el texto principal de las páginas y eliminar elementos no relevantes para la recuperación, como menús de navegación, pies de página, scripts o bloques puramente visuales.

El proceso de obtención del corpus se planteó como una recopilación acotada y reproducible de fuentes relacionadas con el caso de uso del asistente institucional. Para ello, se partió de un conjunto inicial de URLs semilla seleccionadas manualmente por su relevancia para consultas habituales del dominio universitario, incluyendo información sobre trámites académicos, prácticas y becas entre otras.

Durante la recopilación se aplicaron criterios de inclusión y exclusión. Se incluyeron las páginas con información institucional textual, estable, verificable y potencialmente útil para responder preguntas del usuario. En cambio, se descartaron páginas que no superaban un umbral mínimo de 80 palabras de contenido útil tras la limpieza del HTML, páginas cuyo contenido que mayormente tenían elementos de navegación, banners o imágenes sin valor informativo. Esta selección permitió construir un corpus más reducido, pero más controlado y coherente con los objetivos experimentales del trabajo.

El proceso de limpieza del HTML tenía como objetivo conservar el contenido principal de cada página. Para ello, se eliminaron etiquetas de navegación, cabeceras, pies de página, barras laterales y bloques de JavaScript o CSS. Se filtraron bloques HTML con atributos de clase o patrones habituales de ruido en sitios web institucionales, como menús, *banners*, gestión de *cookies*, eventos, contenido social o elementos promocionales. La descarga se realizó respetando las políticas de acceso de cada sitio e introduciendo pausas entre peticiones para no sobrecargar los servidores.

Como resultado, de las 30 páginas descargadas correctamente, 4 fueron descartadas: tres no alcanzaron el umbral mínimo de palabras tras la limpieza y una presentó contenido mayoritariamente visual o estructural sin texto útil recuperable. Cada documento recuperado conserva metadatos asociados a su fuente original, como la URL, el título, la categoría temática y el identificador del documento. Estos metadatos permiten posteriormente relacionar cada fragmento recuperado con su origen, analizar errores de recuperación y mostrar las fuentes utilizadas en las respuestas generadas por el asistente.

Elemento	Valor
URLs semilla consideradas	30
Páginas descargadas correctamente	30
Páginas descartadas durante la limpieza	4
Documentos finales incorporados al corpus	26
Formato final de almacenamiento	Markdown
Tamaño medio de documento	1.557 palabras
<i>Chunks</i> generados	520
Tamaño medio de <i>chunk</i>	851 caracteres

Tabla 3: Resumen del proceso de construcción del corpus documental.

4.7.2. Normalización, segmentación y metadatos

Las páginas descargadas se convierten a Markdown mediante `markdownify` (Anand, 2026), lo que permite obtener una representación textual sencilla y legible, eliminando los elementos propios de la presentación web sin valor informativo. El resultado son ficheros `.md` enriquecidos con los metadatos en la cabecera (título, URL de origen e identificador de documento) que facilitan la trazabilidad durante la recuperación.

A continuación, los documentos se dividen en *chunks*. Esta segmentación es necesaria porque los modelos de embeddings y los modelos generativos trabajan con límites de contexto, y porque la recuperación resulta más precisa cuando se indexan unidades de información de tamaño controlado en lugar de documentos completos.

En este proyecto, la segmentación sigue una estrategia jerárquica orientada a preservar la coherencia semántica de los textos. El documento se divide por límites de párrafo, identificados como bloques separados por una o más líneas en blanco. Si un párrafo supera el tamaño máximo de fragmento establecido (900 caracteres), se aplica una segunda división por oraciones mediante el reconocimiento de signos de puntuación terminal. En caso de que una oración individual resulte todavía demasiado larga, se recurre a una división por palabras como último recurso. Los fragmentos resultantes se construyen acumulando unidades hasta alcanzar el tamaño objetivo de 800 caracteres, con un margen entre 600 y 900 caracteres. Para evitar la pérdida de contexto entre fragmentos consecutivos, se aplica un solapamiento de 150 caracteres, de modo que el final de un fragmento se reutiliza como inicio del siguiente.

Esta estrategia jerárquica se prefiere frente a alternativas más simples por varias razones. La división por tamaño fijo de caracteres o *tokens*, pese a ser la opción más directa, produce cortes arbitrarios que pueden interrumpir una oración, una enumeración o un concepto compuesto, lo que perjudica tanto la representación semántica del *embedding* como la coherencia del contexto proporcionado al modelo generativo. La división únicamente por oraciones genera

fragmentos excesivamente pequeños que pueden carecer de contexto suficiente para que el recuperador comprenda el significado completo del pasaje. La segmentación únicamente por párrafos, sin control del tamaño máximo, puede producir fragmentos demasiado largos que superen la ventana de contexto útil del modelo de *embeddings* y diluyan la señal semántica en la representación vectorial. La estrategia adoptada busca un equilibrio entre estas restricciones, preservando la coherencia textual siempre que sea posible dentro de los límites de tamaño impuestos.

Además, cada fragmento conserva metadatos asociados a su documento de origen, lo que permite mantener la trazabilidad de las respuestas y recuperar la fuente original de la información utilizada por el sistema.

4.8. Almacenamiento vectorial

En los sistemas RAG, el almacenamiento vectorial permite guardar las representaciones numéricas de los fragmentos documentales y recuperarlos posteriormente mediante búsqueda por similitud semántica. Para esta función se ha seleccionado Qdrant (Qdrant, 2026), una base de datos vectorial orientada a la búsqueda semántica y al almacenamiento de vectores junto con información adicional o *payload*.

Qdrant se puede ejecutar de forma local mediante Docker, lo que encaja con el enfoque de bajo coste y control del entorno planteado. Además ofrece persistencia de datos, filtrado por metadatos y una API de consulta que facilita su integración con el pipeline RAG. Estas características resultan especialmente útiles cuando se desea recuperar fragmentos no solo por similitud vectorial, sino también teniendo en cuenta información asociada a la fuente, el documento o la categoría del contenido.

Se valoraron otras alternativas ampliamente utilizadas en el ámbito de la recuperación vectorial. FAISS (Meta AI, 2026) constituye una opción eficiente para búsqueda de similitud y clustering sobre vectores densos, pero se plantea principalmente como una librería y no como una base de datos vectorial completa con servicio persistente y API propia. Chroma (Chroma, 2026), por su parte, ofrece una solución sencilla para almacenar embeddings, realizar búsquedas vectoriales y filtrar por metadatos. Sin embargo, Qdrant se ajusta mejor a las necesidades del proyecto al proporcionar una separación más clara entre el servicio de almacenamiento vectorial y el resto de componentes del sistema.

Por otro lado, Pinecone (Pinecone, 2026) ofrece una base de datos vectorial gestionada orientada a aplicaciones de IA en producción, pero requiere conectividad a un servicio externo de pago, lo que lo descarta para un despliegue local y privado. Aunque Weaviate (Weaviate,

2026) proporciona una base de datos vectorial de código abierto con capacidades de búsqueda semántica e híbrida, exige ejecutar un servidor independiente con un consumo de recursos notable, y no dispone de un modo embebido que permita instanciarlo directamente desde Python sin levantar un proceso adicional.

La Tabla 4 resume las principales características de las alternativas consideradas.

Sistema	Tipo	Persistencia	API propia	Ejecución local
Qdrant	BD vectorial	Sí	Sí	Sí (Docker)
FAISS	Librería de indexación	Manual	No	Sí
Chroma	BD vectorial ligera	Sí	Sí	Sí
Pinecone	BD vectorial gestionada	Sí (cloud)	Sí	No
Weaviate	BD vectorial OSS	Sí	Sí	Sí

Tabla 4: Comparativa de bases de datos vectoriales consideradas.

4.9. Modelos de representación semántica

Los modelos de representación semántica permiten transformar consultas y fragmentos documentales en vectores numéricos, de forma que puedan compararse mediante medidas de similitud. Esta representación es uno de los componentes principales de una arquitectura RAG, ya que determina en gran medida qué información será recuperada y utilizada posteriormente por el modelo generativo.

4.9.1. BGE-M3

Para la generación de embeddings se ha seleccionado BGE-M3 (Chen y col., 2024a), publicado por el Beijing Academy of Artificial Intelligence (BAAI) e integrado mediante la librería `SentenceTransformers`. Este modelo ha sido diseñado para tareas de recuperación multilingüe y destaca por su triple enfoque: *multi-lingual* (soporte para más de cien idiomas), *multi-functionality* (capacidad para producir representaciones densas, dispersas y multi-vector) y *multi-granularity* (ventana de contexto de hasta 8192 tokens).

Desde el punto de vista arquitectónico, BGE-M3 utiliza XLM-RoBERTa-Large como columna vertebral, lo que resulta en aproximadamente 570 millones de parámetros. Su salida por defecto es un vector denso de 1024 dimensiones que captura el significado semántico del texto de entrada y puede compararse mediante similitud coseno con los vectores de los fragmentos del corpus.

Antes de seleccionar BGE-M3 se evaluaron otras alternativas.

- **multilingual-e5-large (Microsoft):** tamaño similar (560M parámetros, 1024 dimensio-

nes) con buen rendimiento multilingüe, pero cobertura en español más limitada en benchmarks de recuperación (Wang y col., 2024).

- ***paraphrase-multilingual-mpnet-base-v2***: modelo más ligero (278M parámetros, 768 dimensiones), con menor capacidad de representación semántica para tareas de recuperación precisas.
- **APIs de OpenAI (*text-embedding-3-small/large*)**: alta calidad, pero con dependencia de servicios externos y coste por uso, incompatible con el enfoque de ejecución local del prototipo.

4.10. Recuperación de información

La recuperación de información es el proceso mediante el cual el sistema selecciona los fragmentos documentales más relevantes para una consulta. En el prototipo se contempla una recuperación densa basada en embeddings y una variante híbrida que combina similitud semántica con recuperación léxica.

4.10.1. Recuperación semántica

La recuperación semántica se realiza generando el embedding de la consulta y comparándolo con los vectores previamente almacenados en la base de datos vectorial. De esta forma, el sistema puede recuperar fragmentos relacionados por significado, aunque no compartan exactamente los mismos términos que la pregunta del usuario. Las consultas en un asistente conversacional suelen formularse de forma natural y diferir de la redacción exacta de los documentos del corpus, por lo que una recuperación de estas características es clave para capturar la intención del usuario y proporcionar contexto relevante al modelo generativo.

En los agentes conversacionales es habitual incorporar una etapa previa de *query rewriting* (Ma y col., 2023), para reformular la consulta del usuario antes de lanzarla contra el índice documental. En este trabajo se implementa una variante de *query rewriting* activable mediante un parámetro de configuración. Cuando está activada, el LLM recibe la pregunta original y produce una consulta reformulada antes de lanzarla contra el índice, mejorando la cobertura en casos donde la formulación del usuario difiere del vocabulario del corpus. En el modo de orquestador ReAct, el propio LLM decide de forma iterativa qué consultas realizar, de modo que la reformulación queda integrada en el bucle de razonamiento y no se aplica como etapa previa independiente.

4.10.2. Recuperación híbrida con BM25

Además de la recuperación densa, el sistema contempla una variante híbrida que combina la similitud vectorial con BM25 (*Best Match 25*), un modelo probabilístico clásico de recuperación léxica basado en coincidencia de términos y ponderación estadística (Robertson y col., 2009).

BM25 asigna a cada documento D una puntuación respecto a una consulta Q con términos $q_1, \dots, q_n$ según la expresión:

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \quad (4.1)$$

donde $f(q_i, D)$ es la frecuencia del término q_i en el documento D , $|D|$ es la longitud del documento en número de términos, avgdl es la longitud media de los documentos de la colección, k_1 y b son hiperparámetros de ajuste (valores típicos: $k_1 = 1,5$, $b = 0,75$), e $\text{IDF}(q_i) = \log \frac{N - n(q_i) + 0,5}{n(q_i) + 0,5}$ penaliza los términos que aparecen en muchos documentos, siendo N el número total de documentos y $n(q_i)$ el número de documentos que contienen el término q_i .

Este mecanismo tiene dos propiedades relevantes para el sistema. El factor IDF premia los términos infrecuentes en la colección, como nombres de trámites, titulaciones o conceptos administrativos específicos, que suelen ser señales discriminativas en consultas institucionales. El factor de normalización por longitud evita que los documentos más largos sean sistemáticamente favorecidos frente a documentos más cortos pero igualmente relevantes.

En la variante híbrida, las listas de resultados de BM25 y de similitud vectorial se fusionan mediante *Reciprocal Rank Fusion* (RRF) (Cormack y col., 2009), una técnica basada en posiciones de ranking en lugar de puntuaciones absolutas. Cada candidato recibe una puntuación proporcional al recíproco de su posición en cada lista ($1/(k + \text{rango})$, con $k = 60$), y las contribuciones de ambas ramas se suman con pesos ajustables (por defecto, 0,5 para cada una). Esta combinación permite equilibrar las ventajas de la búsqueda semántica —que captura relaciones por significado incluso sin coincidencia léxica— con la precisión de BM25 ante términos específicos, mejorando la robustez del sistema ante distintos tipos de preguntas.

4.11. Mejora de la recuperación mediante reranking

El reranking se introduce como una segunda etapa de recuperación. Tras obtener un conjunto inicial de fragmentos candidatos mediante recuperación semántica o híbrida, un modelo adicional vuelve a ordenar dichos fragmentos en función de su relevancia respecto a la con-

sulta. Esta etapa permite corregir el orden inicial cuando los métodos de primera etapa no han situado los fragmentos más relevantes en las posiciones superiores del ranking.

Los modelos de reranking pueden clasificarse en tres categorías según su estrategia de puntuación (Liu, 2009):

- **Pointwise:** asignan una puntuación independiente a cada par (consulta, fragmento), tratando el problema como una tarea de clasificación o regresión sobre ejemplos individuales.
- **Pairwise:** comparan fragmentos de dos en dos, aprendiendo a determinar cuál es más relevante en cada comparación.
- **Listwise:** optimizan directamente la ordenación de toda la lista de candidatos de forma conjunta.

El enfoque *pointwise* se elige frente a *pairwise* y *listwise* por tres motivos: asigna una puntuación absoluta a cada fragmento sin depender de las comparaciones entre candidatos, lo que simplifica la integración en el pipeline; su complejidad escala linealmente con el número de candidatos, siendo más eficiente que *listwise* para listas de tamaño moderado; y dispone de modelos preentrenados de alta calidad ejecutables localmente sin necesidad de entrenamiento adicional.

Dentro del enfoque *pointwise*, existen tres familias principales de arquitecturas para esta tarea, con distintos compromisos entre precisión y coste computacional:

- **Bi-encoder:** codifica la consulta y el fragmento de forma independiente en vectores de embeddings y calcula su similitud coseno. Es la arquitectura utilizada en la primera etapa de recuperación, donde la eficiencia es crítica para escalar a miles de fragmentos. Sin embargo, al no modelar las interacciones directas entre los tokens de la consulta y el fragmento, su capacidad discriminativa es inferior para listas cortas de candidatos.
- **Cross-encoder:** recibe el par (consulta, fragmento) concatenado como una única secuencia y aplica atención conjunta sobre ambos textos, produciendo una puntuación escalar de relevancia. Captura interacciones finas a nivel de token que el bi-encoder no puede modelar, lo que lo hace más preciso para reordenar listas reducidas. Su coste es mayor (requiere una inferencia por par), pero esto es asumible cuando la lista de candidatos ya está acotada a decenas de fragmentos.
- **Late interaction (ColBERT):** codifica consulta y fragmento de forma independiente, pero conserva las representaciones por token para calcular la relevancia mediante operacio-

nes de similitud máxima (*MaxSim*) en tiempo de recuperación. Ofrece un punto intermedio entre bi-encoder y cross-encoder, pero introduce mayor complejidad de implementación y almacenamiento que no está justificada para el tamaño de candidatos de este prototipo.

Se selecciona el *cross-encoder* porque, una vez que la primera etapa de recuperación reduce el espacio de búsqueda a un conjunto pequeño de candidatos, la latencia adicional es despreciable frente a la ganancia en precisión al discriminar fragmentos semánticamente próximos pero con distinta relevancia real.

4.11.1. Modelo de reranking seleccionado

El modelo seleccionado es *BAAI/bge-reranker-v2-m3* (Chen y col., 2024a), publicado por el Beijing Academy of Artificial Intelligence (BAAI), diseñado específicamente para tareas de reranking multilingüe e integrado mediante *SentenceTransformers* (Sentence Transformers, 2026). Se trata de un modelo transformer de tipo encoder, entrenado mediante aprendizaje contrastivo sobre pares de consultas y pasajes relevantes e irrelevantes en múltiples idiomas. El modelo puede ejecutarse localmente y ofrece soporte para las lenguas del corpus, lo que lo hace adecuado para este trabajo.

La Figura 3 ilustra el flujo completo de recuperación híbrida con reranking, mostrando cómo la consulta se procesa en paralelo por ambas ramas antes de fusionarse y reordenarse.

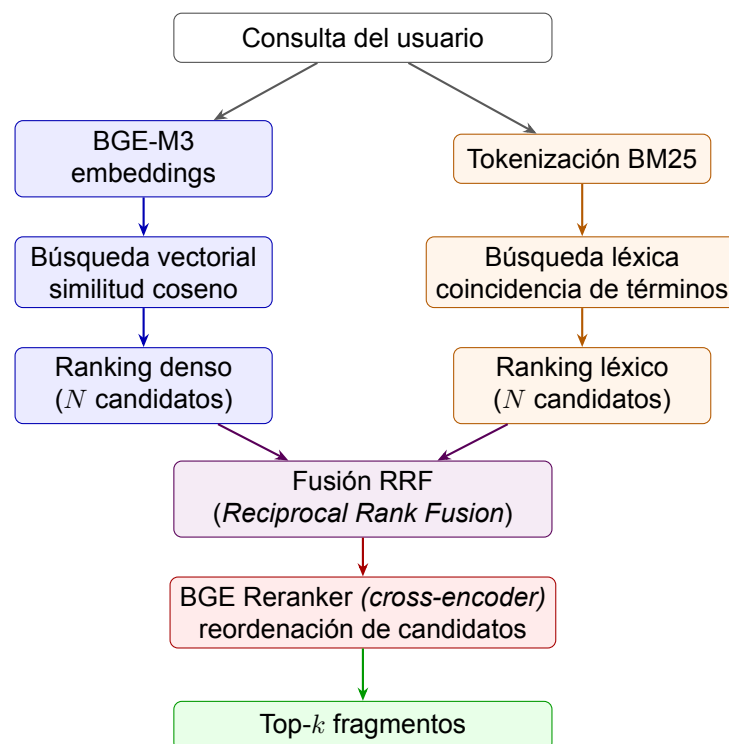


Figura 3: Pipeline de recuperación híbrida: BGE-M3 y BM25 se fusionan mediante RRF antes de ser reordenadas por el reranker.

4.12. Modelo de lenguaje

El modelo de lenguaje es el componente encargado de generar la respuesta final a partir de la pregunta del usuario y del contexto recuperado. En este proyecto, el pipeline se diseña de forma desacoplada respecto al proveedor concreto del modelo, de manera que sea posible utilizar distintas alternativas sin modificar la arquitectura general del sistema.

Durante el desarrollo del prototipo se prioriza el uso de proveedores compatibles con interfaces de tipo API, lo que facilita alternar entre modelos locales y servicios externos. En particular, herramientas como Ollama permiten ejecutar modelos localmente y ofrecen compatibilidad con interfaces similares a la API de OpenAI, lo que simplifica la integración con aplicaciones ya preparadas para este tipo de proveedores (Ollama, 2026). De forma complementaria, vLLM se emplea como servidor de inferencia local compatible con la misma interfaz, lo que permite aprovechar aceleración por GPU cuando está disponible (Kwon y col., 2023).

El modelo utilizado en los experimentos de este trabajo es `qwen2.5:7b` (Qwen Team, 2025), ejecutado localmente mediante Ollama, un modelo instruccional de la familia Qwen 2.5 con 7.000 millones de parámetros, seleccionado por su capacidad para seguir instrucciones en español e inglés y su viabilidad en entornos de cómputo limitados (GPU T4). La arquitectura desacoplada del pipeline permite sustituir este modelo por alternativas de mayor capacidad —como Llama, Mistral o modelos propietarios accesibles mediante API— sin modificar el resto de componentes.

Respecto al orquestador, este sigue el paradigma ReAct (Yao y col., 2023) (*Reasoning + Acting*), en el que el modelo de lenguaje actúa como motor de decisión en un bucle iterativo. En cada iteración, el LLM produce un bloque estructurado compuesto por un pensamiento (*Pensamiento*), la herramienta que decide invocar (*Herramienta*) y su argumento (*Entrada*). El sistema ejecuta la herramienta indicada —búsqueda en el corpus, redacción de correo, rechazo o respuesta final— y devuelve la observación resultante al modelo, que la incorpora en el siguiente paso. Este ciclo *Pensamiento* → *Acción* → *Observación* se repite hasta que el LLM invoca la acción *responder*, indicando que dispone de evidencia suficiente, o la acción *rechazar*, cuando no encuentra información relevante en el corpus. La generación de la respuesta final se delega siempre al prompt RAG estándar, de modo que el orquestador decide *qué* recuperar y *cuándo* responder, mientras que el componente generativo base se encarga de la redacción.

4.13. Interfaz de usuario y observabilidad

La interfaz de usuario y la observabilidad son componentes de apoyo dentro del prototipo. Aunque no constituyen el núcleo del sistema RAG, resultan necesarias para facilitar la interacción con el asistente y analizar el comportamiento del pipeline durante las pruebas.

4.13.1. Streamlit

Para la interfaz conversacional se ha seleccionado Streamlit (Streamlit, 2026), un framework de código abierto orientado a la creación de aplicaciones interactivas en Python. Su principal ventaja en este proyecto es que permite construir una interfaz funcional sin introducir una capa de desarrollo frontend compleja.

Esta elección resulta adecuada porque el objetivo principal del trabajo no es el diseño de una interfaz avanzada, sino la implementación y evaluación de una arquitectura RAG con recuperación, reranking y orquestación. Por ello, Streamlit permite disponer de un entorno de prueba accesible para interactuar con el sistema, visualizar respuestas y comprobar el funcionamiento del pipeline de forma rápida.

Además, la interfaz se mantiene desacoplada del núcleo del sistema. Esta separación permite modificar o sustituir la capa visual en el futuro sin afectar a los componentes principales de ingesta, recuperación, generación u orquestación. La Figura 4 muestra el aspecto de la interfaz resultante.

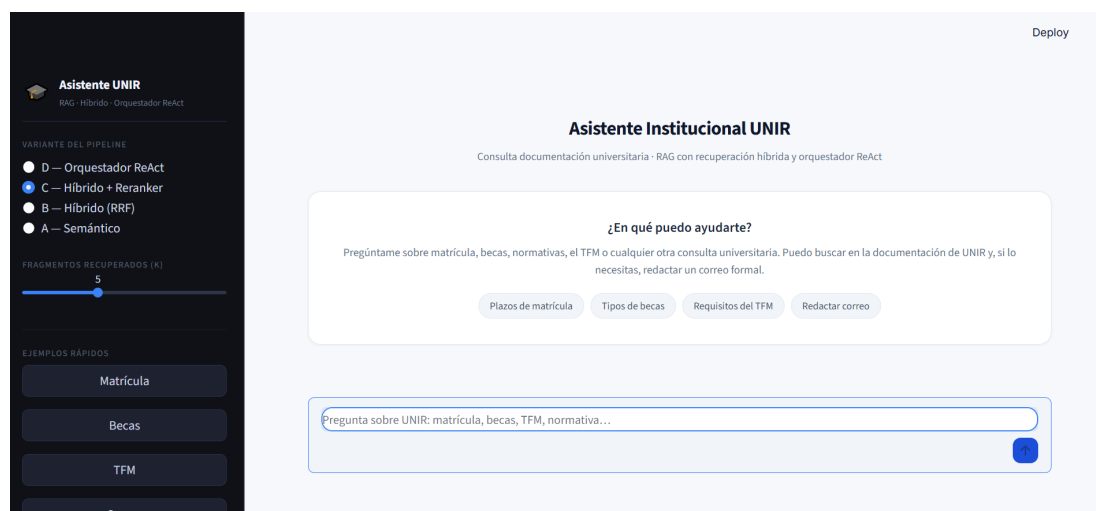


Figura 4: Interfaz conversacional del asistente implementada con Streamlit.

4.13.2. Phoenix

Para la observabilidad del sistema se utiliza Phoenix (Arize AI, 2026), una plataforma de código abierto de Arize AI orientada al análisis, depuración y evaluación de aplicaciones basa-

das en modelos de lenguaje. Phoenix proporciona una interfaz web que permite visualizar en tiempo real el comportamiento interno del pipeline: desde la consulta original hasta la respuesta final, pasando por cada etapa intermedia.

La integración se implementa mediante OpenTelemetry (OpenTelemetry, 2026), un framework abierto y estándar de la industria para la generación, recopilación y exportación de datos de telemetría (trazas, métricas y registros). Un TracerProvider configurado con un OTLPSpanExporter exporta las trazas al servidor Phoenix desplegado en Docker mediante el protocolo gRPC. Esta arquitectura desacopla, si en el futuro se sustituyera Phoenix por otra plataforma compatible con OpenTelemetry (como Jaeger o Grafana Tempo), no sería necesario modificar el código de la aplicación.

Cada consulta genera una traza que recorre el pipeline de principio a fin. A nivel global se registran la pregunta del usuario, la configuración activa (modo de recuperación, uso de reranking, valor de k) y la latencia total. Dentro de esa traza, cada etapa registra su propia información:

- **Recuperación:** qué documentos devolvió el sistema, en qué orden y con qué puntuación de similitud, junto con un extracto de cada fragmento.
- **Reranking** (variantes C y D): las nuevas puntuaciones asignadas por el *cross-encoder* tras reordenar los candidatos, lo que permite comprobar si el reranker mejora la ordenación.
- **Generación:** el prompt completo que recibió el modelo, la respuesta que produjo y el número de tokens consumidos.
- **Reescritura de consulta** (cuando está activa): la pregunta original y la versión reformulada usada para la recuperación.

Durante el desarrollo, Phoenix permitió detectar fallos de recuperación y prompts mal formados de forma inmediata, así como controlar el consumo de tokens por consulta, sin necesidad de depuración manual. Durante la evaluación experimental, facilitó verificar que las diferencias de rendimiento entre variantes se debían a los componentes evaluados y no a artefactos de configuración. En producción, la misma capa aportaría valor adicional para monitorizar latencias y mantener trazabilidad auditable sobre las respuestas generadas.

Capítulo 5

Resultados experimentales

Este capítulo presenta los resultados obtenidos durante la evaluación del asistente institucional desarrollado. En primer lugar, se describe brevemente el corpus utilizado, el dataset de evaluación y las métricas empleadas. A continuación, se comparan los resultados de recuperación del baseline determinista frente a la variante con recuperación híbrida y reranking. Posteriormente, se analiza la calidad de las respuestas del flujo determinista y del orquestador. Finalmente, se discuten los resultados obtenidos, destacando las mejoras observadas y las principales limitaciones del proceso de evaluación.

5.1. Diseño de la evaluación

La evaluación experimental se diseñó con el propósito de comparar el comportamiento del sistema en diferentes configuraciones. Para ello, se utilizó un corpus documental construido a partir de información pública de UNIR y un conjunto de preguntas diseñado específicamente para cubrir distintos tipos de consulta: preguntas respondibles desde un único documento, preguntas que requieren combinar información de varias fuentes, preguntas comparativas y preguntas no respondibles a partir del corpus disponible.

5.1.1. Corpus y dataset de evaluación

La evaluación se realizó sobre un corpus documental construido a partir de información pública del dominio institucional. Tras el proceso de recopilación, limpieza, normalización y segmentación descrito en el capítulo anterior, el corpus final quedó formado por 26 documentos en formato Markdown. Este formato se seleccionó porque permite conservar una estructura textual sencilla, facilitar la segmentación por bloques de contenido y mantener una representación homogénea de fuentes originalmente heterogéneas.

A partir de este corpus se construyó manualmente un dataset de evaluación compuesto por 71 preguntas representativas del dominio. El objetivo de este conjunto no es constituir un benchmark general de asistentes institucionales, sino proporcionar una validación controlada del prototipo y permitir la comparación de distintas variantes del sistema bajo las mismas condiciones experimentales.

Las preguntas se diseñaron buscando cubrir escenarios habituales de uso en un contexto universitario. En concreto, se incluyeron preguntas respondibles desde un único documento, preguntas que requieren combinar información procedente de varias fuentes, preguntas comparativas y preguntas no respondibles a partir del corpus disponible. Esta última categoría se incorporó para evaluar la capacidad del sistema de abstenerse cuando no existe evidencia suficiente, para no afectar en la confianza del usuario.

Para las preguntas respondibles se anotó manualmente la fuente esperada, es decir, el documento o documentos en los que debía encontrarse la evidencia necesaria para responder. Todas las preguntas respondibles del conjunto final cuentan con al menos dos hechos esperados anotados. Esta anotación permitió evaluar no solo si el sistema recuperaba la fuente adecuada, sino también si la respuesta generada incorporaba los elementos informativos relevantes.

El tamaño del dataset es de 71 preguntas, los valores obtenidos deben entenderse como una aproximación a la evaluación del sistema y no como una medición definitiva, dado que el corpus documental cubre exclusivamente el dominio público de UNIR. Por tanto, los resultados deben interpretarse como una evaluación inicial y controlada, no como una medición definitiva del rendimiento del sistema en producción.

Elemento	Valor
Documentos finales del corpus	26
Formato de almacenamiento	Markdown
Tamaño medio de documento	1557,92 palabras
Preguntas totales del dataset	71
Preguntas respondibles	61
Preguntas no respondibles	10
Preguntas con fuente esperada anotada	61
Preguntas con hechos esperados anotados	61

Tabla 5: Resumen del corpus y del dataset utilizados en la evaluación.

Estos valores reflejan un proceso de recopilación acotado y controlado. Aunque el número de documentos finales no es elevado, el corpus se construyó a partir de fuentes seleccionadas manualmente y revisadas durante la limpieza, priorizando la relevancia institucional, la trazabilidad de la fuente y la utilidad para el conjunto de preguntas de evaluación. Por tanto, el objetivo no es maximizar el volumen documental, sino disponer de una base suficientemente represen-

tativa y coherente para validar el comportamiento del prototipo bajo condiciones comparables.

Tipo de pregunta	Número de preguntas
Responsable desde un único documento	36
Responsable con combinación de documentos	15
Comparativa	10
No responsable	10

Tabla 6: Distribución de preguntas del dataset de evaluación.

La Tabla 7 muestra un ejemplo representativo de cada tipo de pregunta, junto con la fuente esperada y un extracto de los hechos anotados, con el objetivo de ilustrar el proceso de construcción y anotación manual del dataset.

Pregunta	Tipo	Fuente esperada	Hechos esperados (extracto)
¿Qué requisitos de acceso se piden para estudiar en UNIR?	Único doc.	requisitos-acceso	(1) Los requisitos dependen del tipo de estudio. (2) Para postgrado oficial se necesita título universitario oficial.
¿Qué pasos debo seguir para matricularme y dónde ver los requisitos de acceso?	Multi-doc.	como-matricularse, requisitos-acceso	(1) El proceso comienza con el formulario de solicitud de información. (2) Un asesor personal guía el proceso.
Compara la oferta de la Facultad de Educación con la de la ESIT de UNIR.	Comparativa	facultad-educacion, esit	(1) La Facultad de Educación ofrece 17 grados centrados en docencia. (2) La ESIT ofrece 9 grados centrados en tecnología.
¿Cuál es la tasa exacta de empleabilidad de cada máster de UNIR en 2025?	No responsable	—	—

Tabla 7: Ejemplos representativos del dataset de evaluación con fuente y hechos esperados anotados.

5.1.2. Métricas empleadas

Para evaluar el sistema se utilizaron métricas diferentes en función del componente analizado. En la fase de recuperación documental se emplearon métricas habituales en sistemas de búsqueda de información:

- **Hit@1**: indica si la fuente esperada aparece en la primera posición de los resultados recuperados.
- **Hit@3**: indica si la fuente esperada aparece entre los tres primeros resultados recuperados.
- **MRR**: mide la posición media recíproca de la primera fuente relevante recuperada.

En todas las variantes evaluadas, el número de fragmentos recuperados y proporcionados al modelo de lenguaje como contexto es $k = 5$. En la variante con recuperación híbrida y reranking, el recuperador obtiene hasta 20 candidatos iniciales (10 densos más 10 léxicos), que el reranker reduce a los 5 más relevantes antes de la generación.

Para evaluar la calidad de las respuestas se utilizaron métricas orientadas al contenido generado:

- **Cobertura del *ground truth* (Fact-Coverage)**: proporción de hechos esperados del *ground truth* que aparecen cubiertos en la respuesta generada, medida mediante solapamiento de tokens.
- **ROUGE-L F1**: mide la subsecuencia común más larga entre la respuesta generada y cada hecho esperado, calculando una media sobre todos los hechos de la pregunta.
- **BERTScore F1**: mide la similitud semántica entre la respuesta generada y los hechos esperados mediante representaciones contextuales del modelo multilingüe *bert-base-multilingual-cased*, calculando una media sobre todos los hechos de la pregunta.
- **Tasa de abstención**: proporción de preguntas no respondibles en las que el sistema evita responder sin evidencia suficiente.
- **Exactitud en la detección de intención de correo**: capacidad del orquestador para identificar solicitudes que requieren activar la herramienta de correo electrónico.

5.1.3. Variantes evaluadas (ablation study)

Las variantes del sistema se diseñaron como un *ablation study*: cada configuración introduce un componente adicional respecto a la anterior, lo que permite aislar el efecto de cada elemento del pipeline sobre las métricas de evaluación. La Tabla 8 resume las cuatro variantes propuestas.

Variante	Recuperación	Reranking	Orquestador
A: Semántico + LLM	Densa	No	No
B: Híbrido + LLM	Densa + léxica	No	No
C: Híbrido + Reranker + LLM	Densa + léxica	Sí	No
D: Orquestador + Híbrido + Reranker + LLM	Densa + léxica	Sí	Sí

Tabla 8: Diseño del *ablation study*: variantes evaluadas.

En el trabajo se evaluaron las cuatro variantes del *ablation study*. Las variantes A, B y C permiten aislar el efecto individual de la recuperación híbrida y del reranker sobre la calidad de la evidencia recuperada. La variante D combina el orquestador con recuperación híbrida y

reranking, y permite analizar el efecto del componente de orquestación sobre el control del flujo y la activación de herramientas.

El orquestador implementado sigue el paradigma ReAct (Yao y col., 2023) (*Reasoning + Acting*), en el que el modelo de lenguaje actúa como motor de decisión dentro de un bucle iterativo de *Pensamiento* → *Acción* → *Observación*. En cada paso, el LLM selecciona la herramienta a invocar y el sistema devuelve la observación al modelo para el siguiente ciclo. Este diseño permite que el orquestador adapte su estrategia de recuperación en tiempo de ejecución en función del contenido real de los documentos encontrados.

5.2. Evaluación sobre dataset privado: UNIR

5.2.1. Resultados de recuperación documental

En esta sección se comparan los resultados obtenidos por las variantes A, B y C del *ablation study*, evaluadas sobre las preguntas respondibles del dataset que cuentan con una fuente esperada, excluyendo las preguntas no respondibles.

Variante	Hit@1	Δ	Hit@3	Δ	MRR	Δ
A: Semántico + LLM	0.574	–	0.852	–	0.712	–
B: Híbrido (RRF) + LLM	0.525	–0.049	0.902	+0.050	0.706	–0.006
C: Híbrido (RRF) + Reranker + LLM	0.557	–0.017	0.918	+0.066	0.738	+0.026
D: Orquestador + Híbrido + Reranker*	0.557	–0.017	0.918	+0.066	0.738	+0.026

* La variante D emplea el mismo retriever que C (híbrido RRF + reranker); los valores de recuperación son idénticos.

Tabla 9: Resultados de recuperación del *ablation study*.

Los resultados muestran un patrón diferenciado según el componente incorporado. La variante A (semántico puro) obtiene Hit@1 = 0.574, Hit@3 = 0.852 y MRR = 0.712 como línea base, con buena precisión en la primera posición pero cobertura moderada en el top-3.

La variante B (recuperación híbrida con *Reciprocal Rank Fusion*) obtiene Hit@1 = 0.525 (–0.049 respecto a A), Hit@3 = 0.902 (+0.050) y MRR = 0.706 (–0.006). La combinación densa y léxica introduce mayor diversidad de candidatos, lo que incrementa la presencia de documentos relevantes en el top-3 (+5,0 puntos porcentuales); el descenso en Hit@1 podría relacionarse con la incorporación de candidatos léxicos que desplazan fragmentos semánticamente más relevantes de la primera posición.

La variante C (híbrido RRF + reranker *BAAI/bge-reranker-v2-m3*) obtiene Hit@1 = 0.557 (–0.017 respecto a A), Hit@3 = 0.918 (+0.066) y MRR = 0.738 (+0.026). El reranker multilingüe recupera parcialmente la precisión en primera posición y eleva la MRR por encima de la línea base, lo que indica que el modelo *cross-encoder* es capaz de reordenar correctamente los

candidatos del pool híbrido. Hit@3 alcanza el valor más alto de las tres variantes (0.918), lo que es coherente con que la combinación de recuperación híbrida con reranking proporcione el pool de evidencia más rico para la generación.

5.2.2. Resultados de calidad de respuesta

Además de evaluar la recuperación documental, se analizaron las respuestas generadas por las cuatro variantes del *ablation study*. Esta evaluación permite comprobar si las mejoras introducidas en el pipeline de recuperación se traducen en respuestas con mayor cobertura informativa.

La evaluación se realizó sobre las 61 preguntas respondibles del dataset, incluyendo la variante D (orquestador), evaluada también sobre las 61 preguntas para garantizar la comparabilidad.

Variante	Fact-Cov.	Δ	ROUGE-L	BERTScore F1
A: Semántico + LLM	0.389	–	0.093	0.643
B: Híbrido + LLM	0.496	+0.107	0.094	0.641
C: Híbrido + Reranker + LLM	0.527	+0.138	0.102	0.656
D: Orquestador + Híbrido + Rer.	0.493	+0.104	0.094	0.648

Δ respecto al *baseline* (variante A).

Tabla 10: Resultados del *ablation study*: calidad de respuesta sobre preguntas respondibles.

Los resultados muestran una progresión consistente a lo largo del *ablation study*. La variante A, con recuperación semántica densa, obtiene una cobertura de hechos esperados de 0.389 y un BERTScore F1 de 0.643. La incorporación de recuperación híbrida en la variante B produce la mejora más notable: la cobertura factual aumenta hasta 0.496 (+0.107, +27,5 % relativo), lo que sugiere que combinar búsqueda densa y léxica permite recuperar fragmentos más informativos. BERTScore permanece prácticamente igual (0.641), lo que sugiere que la ganancia se concentra en la cobertura léxica de los hechos esperados más que en la similitud semántica global de la respuesta.

La variante C añade el reranker multilingüe y produce mejoras simultáneas en las tres métricas: Fact-Coverage sube a 0.527 (+0.138 respecto al baseline), ROUGE-L a 0.102 y BERTScore a 0.656. La mejora conjunta en las tres métricas sugiere que el reranker selecciona fragmentos de mayor calidad informativa, no solo léxica, lo que podría reflejarse en respuestas más completas y semánticamente más alineadas con los hechos esperados.

La variante D (orquestador ReAct con recuperación híbrida y reranking), evaluada sobre las mismas 61 preguntas que el resto de variantes, obtiene una Fact-Coverage de 0.493 (+0.104 respecto al baseline A), ROUGE-L de 0.094 y BERTScore de 0.648. C sigue siendo la variante

con mayor cobertura factual (0.527), y D se sitúa ligeramente por debajo en métricas léxicas pero mantiene un BERTScore competitivo (0.648 frente a 0.656 de C, diferencia de 0.008). Este resultado es coherente con la naturaleza del flujo ReAct: el orquestador tiende a generar respuestas más elaboradas y estructuradas que no maximizan el solapamiento léxico con los *expected facts*, pero mantienen una calidad semántica equivalente. El orquestador no está diseñado para mejorar la calidad textual de las respuestas informativas, sino para ampliar las capacidades del sistema en dimensiones que estas métricas no capturan; dichas capacidades se analizan en la sección 5.2.4.

Respecto al comportamiento ante preguntas no respondibles, todas las variantes fueron evaluadas sobre las 10 preguntas de este tipo presentes en el dataset. Las tasas de abstención obtenidas fueron 0.500 (A), 0.300 (B), 0.400 (C) y 0.600 (D). Estos valores muestran que el modelo *qwen2.5:7b* presenta una capacidad de abstención innata incluso en los flujos deterministas. La variante D, al incorporar el orquestador ReAct, mejora la abstención hasta 0.600 gracias al mecanismo explícito de rechazo. Además un experimento de sensibilidad modificando la condición de rechazo del orquestador evaluó la relevancia del contenido recuperado y no solo su presencia: la tasa de abstención aumentó a 0.800, aunque se observó un incremento del rechazo incorrecto en algunas categorías (sobre-rechazo), con una caída de Fact-Coverage a 0.430.

5.2.3. Análisis cualitativo de casos representativos

Las métricas agregadas reflejan el comportamiento medio del sistema, pero no permiten apreciar por qué una respuesta semánticamente correcta puede obtener una Fact-Coverage baja, ni en qué difiere el estilo generativo del orquestador frente al flujo determinista. Las siguientes tablas presentan tres casos concretos que ilustran estos fenómenos.

Campo	Contenido
Pregunta	¿Existe una bolsa de empleo para estudiantes en UNIR?
Variante	C: Híbrido + Reranker + LLM
Fuente esperada	bolsa-de-empleo-coie
Fragmento recuperado (top-1)	«Como estudiante de UNIR, tendrás acceso directo y gratuito a la bolsa de empleo de UNIR y a múltiples programas de empleo.»
Respuesta generada	«Sí, existe una bolsa de empleo gestionada a través del COIE. Los estudiantes tienen acceso a más de 2.200 empresas registradas y el acceso es directo y gratuito.»
Hechos cubiertos	El COIE gestiona la bolsa (✓); más de 2.200 empresas registradas (✓); acceso gratuito (✓).
Hechos no cubiertos	Prácticas en más de 1.500 empresas (×): el dato estaba en un fragmento de menor rango no incluido en los 5 finales.
Comentario	Éxito parcial. La fuente esperada es recuperada en primera posición. La respuesta cubre 3 de 4 hechos; la omisión ilustra cómo la generación puede no incorporar hechos presentes en fragmentos de bajo rango.

Tabla 11: Ejemplo cualitativo 1: pregunta de un documento con éxito parcial (variante C).

Campo	Contenido
Pregunta	¿Cómo es el Campus Virtual de UNIR?
Hechos esperados	(1) Accesible desde cualquier dispositivo; (2) Aulas virtuales con clases en directo y diferido; (3) Agenda de actividades y exámenes; (4) Acceso a Biblioteca digital.
Respuesta variante C	«[...] Asistencia a clases en vivo. Clases en diferido: todas las sesiones quedan guardadas [...]» (585 caracteres, 2 elementos relevantes).
Respuesta variante D	«[...] Asistencia a clases virtuales en vivo [...] Gestión de actividades: los estudiantes pueden visualizar y gestionar tareas y calificaciones [...]» (909 caracteres, mayor estructuración).
Hechos cubiertos	C: (2) ✓; D: (2) ✓, (3) ✓ (parcial). Ambas variantes omiten (1) y (4).
Comentario	D genera una respuesta más elaborada que C, pero no cubre significativamente más hechos esperados. BERTScore se mantiene estable en ambas (contenido semánticamente relacionado), mientras que Fact-Coverage no mejora porque los hechos específicos no se enuncian de forma explícita. Ilustra por qué FC puede bajar con el orquestador a pesar de una respuesta aparentemente más completa.

Tabla 12: Ejemplo cualitativo 2: elaboración del orquestador sin mejora de Fact-Coverage (variante C vs. D).

Campo	Contenido
Pregunta	¿Cuál es la tasa exacta de empleabilidad de cada máster de UNIR en 2025?
Tipo	No respondible (información no presente en el corpus).
Fragmento recuperado (top-1)	«El 80 % de los estudiantes de la Facultad de Derecho mejoran profesionalmente [...]» (dato general, no específico por máster ni por año 2025).
Respuesta (variante C)	«No tengo información sobre la tasa exacta de empleabilidad de cada máster de UNIR para el año 2025. El contexto proporcionado no incluye datos específicos sobre esta métrica.»
Hechos cubiertos / no cubiertos	No aplicable: la abstención es la respuesta correcta.
Comentario	Abstención correcta. El sistema reconoce que los fragmentos recuperados no contienen la información específica solicitada y evita generar una respuesta. La variante D incorpora además un mecanismo explícito de rechazo en el orquestador, lo que eleva la tasa de abstención de 0.400 (C) a 0.600 (D) sobre las preguntas no respondibles del dataset.

Tabla 13: Ejemplo cualitativo 3: abstención ante pregunta no respondible.

5.2.4. Evaluación funcional del orquestador

La evaluación del orquestador no se limita únicamente a métricas de respuesta, ya que su objetivo principal no es sustituir al recuperador, sino coordinar el flujo de ejecución del asistente. Por ello, se realizaron pruebas funcionales orientadas a comprobar si el sistema identifica correctamente el tipo de consulta y selecciona la acción adecuada.

La Tabla 14 resume los resultados obtenidos en los principales escenarios, expresados mediante métricas objetivas extraídas de los registros de evaluación.

Tipo de consulta	n	Enrutamiento correcto	Llamadas herramienta (media)	Abstención
Un documento	15	13/15 (0,87)	1,0	0/15 (0,00)
Multi-documento	6	2/6 (0,33)	1,3	0/6 (0,00)
Comparativa	4	4/4 (1,00)	1,8	0/4 (0,00)
No respondible	5	—	—	5/5 (1,00)
Solicitud de correo	6	5/6 (0,83)	—	—

Tabla 14: Evaluación funcional del orquestador con métricas objetivas (n = número de preguntas de cada tipo).

Los resultados muestran un comportamiento heterogéneo según el tipo de consulta. Las preguntas comparativas alcanzan el enrutamiento perfecto (4/4), y las de un único documento obtienen un 87 % de enrutamiento correcto con una media de una llamada a herramienta por consulta. Las preguntas no respondibles son gestionadas correctamente en todos los casos, con una tasa de abstención del 100 %. Las solicitudes de correo se detectan en 5 de 6 casos (83 %). Sin embargo, el enrutamiento de preguntas multi-documento resulta el más problemático, con solo un 33 % de clasificaciones correctas, lo que sugiere que este tipo de consulta resulta especialmente difícil de identificar de forma automática y requeriría mayor esfuerzo de

ajuste en iteraciones futuras.

5.3. Evaluación sobre dataset público: XQuAD-es

Con el objetivo de verificar que los componentes de recuperación generalizan más allá del corpus institucional de UNIR, se realizó una evaluación adicional sobre el dataset público **XQuAD-es** (Artetxe y col., 2020), la variante en español del benchmark *Cross-lingual Question Answering Dataset*. XQuAD-es consta de 1.190 pares pregunta-respuesta distribuidos sobre 240 párrafos extraídos de la Wikipedia en español, y es ampliamente utilizado como referencia de recuperación y comprensión lectora en lenguas distintas del inglés.

La evaluación de recuperación se realizó sobre los 1.190 pares completos, con los 240 párrafos codificados mediante el mismo modelo de embeddings (*BAAI/bge-m3*) empleado en el corpus de UNIR. Para la variante D, los párrafos se ingirieron en una colección Qdrant temporal e independiente, generando 502 fragmentos, con el mismo pipeline de ingesta del sistema. La evaluación de generación se realizó sobre un subconjunto aleatorio de 25 preguntas por variante, seleccionado con semilla fija para garantizar la comparabilidad; evaluar las 1.190 preguntas era inviable por el coste computacional de las llamadas al modelo de lenguaje.

5.3.1. Resultados de recuperación

La Tabla 15 muestra los resultados de recuperación de las variantes A, B y C sobre el dataset completo de XQuAD-es.

Variante	Hit@1	Δ	Hit@3	Δ	MRR	Δ
A: Semántico	0.930	–	0.978	–	0.956	–
B: Híbrido (RRF)	0.921	–0.009	0.984	+0.006	0.953	–0.003
C: Híbrido + Reranker	0.984	+0.054	0.999	+0.021	0.991	+0.035

Tabla 15: Resultados de recuperación sobre XQuAD-es (1.190 preguntas, 240 contextos únicos).

El patrón es idéntico al observado en la evaluación interna: C obtiene los mejores resultados en todas las métricas, mientras que B presenta un ligero descenso en Hit@1 respecto a A (–0.009) aunque mejora en Hit@3 (+0.006). Este comportamiento es coherente con la naturaleza de XQuAD: las preguntas se formulan mediante paráfrasis del texto original y no comparten vocabulario directo con los pasajes, lo que reduce la utilidad del componente léxico (BM25) en la primera posición. El reranker *cross-encoder* recupera esta pérdida de forma efectiva, elevando Hit@1 en 5.4 puntos porcentuales respecto a A y situando Hit@3 en 0.999 (una sola pregunta de 1.190 sin el pasaje correcto en las tres primeras posiciones).

Los valores absolutos son más elevados que en la evaluación interna, lo que obedece a dos factores: el corpus XQuAD es reducido (240 pasajes) y las preguntas fueron diseñadas explícitamente a partir de esos mismos pasajes, lo que maximiza la alineación semántica entre pregunta y documento. Estas condiciones difieren del escenario real de UNIR, donde el corpus es más heterogéneo y las consultas de los usuarios no guardan la misma exactitud con los documentos.

5.3.2. Resultados de generación

La Tabla 16 recoge los resultados de calidad de respuesta de las cuatro variantes evaluadas sobre el corpus XQuAD temporal (25 preguntas por variante).

Variante	Fact-Coverage	ROUGE-L F1	BERTScore F1
A: Semántico + LLM	0.280	0.075	0.574
B: Híbrido (RRF) + LLM	0.280	0.072	0.575
C: Híbrido + Reranker + LLM	0.200	0.055	0.563
D: Orquestador + Híbrido + Reranker	0.160	0.046	0.568

Tabla 16: Resultados de generación sobre XQuAD-es (corpus XQuAD temporal, 25 preguntas por variante).

El primer resultado destacable es que el **BERTScore F1 es muy estable en todas las variantes** (0.563–0.575), lo que indica que la calidad semántica de las respuestas generadas no varía significativamente entre configuraciones. Esta estabilidad es coherente con la naturaleza de XQuAD: el corpus es reducido (240 pasajes únicos) y cualquier variante que recupere el pasaje correcto —algo que ocurre con alta probabilidad dadas las métricas de recuperación— genera una respuesta semánticamente equivalente.

La cobertura factual y ROUGE-L muestran un patrón diferente al observado en la evaluación interna. A y B obtienen la misma Fact-Coverage (0.280), mientras que C desciende a 0.200 y D a 0.160. Una posible explicación es la naturaleza del benchmark: XQuAD define las respuestas esperadas como fragmentos exactos y cortos del texto original (por ejemplo, un nombre, una fecha o una cifra), mientras que el sistema genera respuestas en lenguaje natural completas, no extrae spans. Cuando la respuesta generada es correcta pero está expresada con palabras distintas a las del fragmento esperado, Fact-Coverage y ROUGE-L la penalizan al medir coincidencia léxica y no semántica. Esta desalineación podría acentuarse en C, donde el reranker puede priorizar fragmentos semánticamente relevantes que no contienen el vocabulario exacto de la respuesta esperada, y en D, donde el orquestador tiende a generar respuestas más elaboradas que se alejan léxicamente del span de referencia.

Un ejemplo concreto ilustra este efecto. Ante la pregunta «¿Cuántos balones interceptó

Josh Norman?», el *expected fact* de XQuAD es el span exacto *cuatro*". La respuesta generada por el sistema fue: «*Josh Norman interceptó 4 pases durante la temporada [2][3]*». La respuesta es factualmente correcta —4 equivale a cuatro—, pero ROUGE-L asigna 0 al no existir solapamiento léxico entre *cuatro*" y "4". BERTScore, en cambio, captura la equivalencia semántica y asigna una puntuación de 0.571, más representativa de la calidad real de la respuesta.

BERTScore, al medir similitud semántica mediante representaciones contextuales en lugar de solapamiento exacto de tokens, es la métrica más adecuada en este contexto. Los valores obtenidos (0.563–0.575) sugieren que todas las variantes generan respuestas de calidad semántica equivalente, y que las diferencias en Fact-Coverage son un artefacto de la métrica y no una degradación real de la calidad de respuesta.

5.3.3. Comparación con la evaluación interna

La Tabla 17 pone en perspectiva los resultados de ambas evaluaciones para las métricas de recuperación disponibles en las dos.

Corpus	Variante	Hit@1	Hit@3	MRR
UNIR (interno)	A: Semántico	0.574	0.852	0.712
	B: Híbrido (RRF)	0.525	0.902	0.706
	C: Híbrido + Reranker	0.557	0.918	0.738
XQuAD-es	A: Semántico	0.930	0.978	0.956
	B: Híbrido (RRF)	0.921	0.984	0.953
	C: Híbrido + Reranker	0.984	0.999	0.991

Tabla 17: Comparación de métricas de recuperación entre la evaluación interna (UNIR) y XQuAD-es.

Más allá de las diferencias en valores absolutos, los resultados de XQuAD apuntan a dos conclusiones: el **ordenamiento relativo entre variantes se mantiene** ($C > A > B$ en Hit@1; $C > B > A$ en Hit@3) y **el reranker aporta la ganancia más consistente** en ambas evaluaciones. Esto sugiere que la arquitectura propuesta no sobreajusta al corpus de UNIR, sino que sus componentes de recuperación muestran un comportamiento coherente en los dos entornos evaluados.

5.4. Discusión de resultados

Los resultados obtenidos permiten extraer varias conclusiones relevantes sobre el comportamiento del sistema.

En primer lugar, la incorporación de recuperación híbrida y reranking mejora el rendimiento del sistema en las métricas de recuperación. La variante B (híbrido RRF) muestra que la fusión léxico-semántica amplía la cobertura en el top-3: Hit@3 pasa de 0.852 a 0.902 (+0.050) respecto a A, aunque Hit@1 desciende ligeramente (−0.049) y MRR se mantiene prácticamente igual (−0.006). Este patrón sugiere que BM25 introduce candidatos léxicamente relevantes que mejoran la presencia de la fuente esperada en posiciones intermedias, pero no siempre en primera posición. La variante C (híbrido RRF + reranker *BAAI/bge-reranker-v2-m3*) obtiene Hit@1 = 0.557 (−0.017 respecto a A), Hit@3 = 0.918 (+0.066) y MRR = 0.738 (+0.026). El reranker multilingüe reordena el pool híbrido de forma más precisa: Hit@3 alcanza su valor máximo y MRR supera al baseline, lo que sugiere que el modelo *cross-encoder* aporta una discriminación adicional que no ofrece ni la búsqueda semántica ni la léxica por separado.

En segundo lugar, la variante D (orquestador) obtiene una cobertura factual de 0.493, ligeramente inferior a C (0.527, −0.034). Este resultado sugiere que la mayor parte de la ganancia en calidad de respuesta proviene del pipeline de recuperación híbrido con reranking, y que el orquestador no añade una mejora en cobertura factual léxica cuando el retriever ya proporciona evidencia de buena calidad. La diferencia en BERTScore es mínima (0.648 vs 0.656), lo que sugiere que la calidad semántica de las respuestas es equivalente y que la penalización en Fact-Coverage es coherente con el estilo más elaborado de las respuestas del flujo ReAct más que con una degradación real. El orquestador aporta valor en dimensiones que van más allá de la cobertura factual: la tasa de abstención del flujo ReAct (0.600) supera a los flujos deterministas (A: 0.500, B: 0.300, C: 0.400), y la detección de solicitudes de correo (0.833 de exactitud) indica que la arquitectura puede extenderse hacia funcionalidades accionables, manteniendo un flujo controlado y trazable.

En conjunto, los resultados muestran que las mejoras sobre el recuperador tienen un impacto directo y medible en la calidad de las respuestas generadas: la cobertura factual pasa de 0.389 (variante A) a 0.527 (variante C), una mejora del 35,5 %, coherente con la progresión observada también en ROUGE-L y BERTScore. El orquestador aporta valor principalmente en términos de control del flujo, abstención ante preguntas no respondibles y activación de capacidades accionables; su Fact-Coverage (0.493) es ligeramente inferior a C debido al estilo más elaborado de sus respuestas, pero su BERTScore (0.648) se mantiene próximo, lo que indica una calidad semántica equivalente.

Los resultados deben interpretarse desde la contribución global de la arquitectura: la mejora del retrieval incrementa la calidad de la evidencia disponible, el reranker selecciona los fragmentos más informativos de un pool de candidatos más diverso, y la orquestación permite

controlar cómo se utiliza dicha evidencia y cuándo debe activarse una capacidad accionable. La aportación principal no reside en un único componente aislado, sino en la integración modular y trazable de estos elementos dentro de un asistente institucional supervisado.

Finalmente, la evaluación sobre XQuAD-es refuerza las conclusiones anteriores desde una perspectiva de generalización. El ordenamiento de variantes se reproduce fielmente en un corpus de dominio diferente: C sigue siendo la configuración de recuperación más robusta ($\text{Hit@1} = 0.984$, $\text{MRR} = 0.991$), y B continúa mostrando el mismo patrón de mayor diversidad en el top-3 a costa de precisión en la primera posición. En cuanto a la calidad de respuesta, el BERTScore es muy estable en todas las variantes (0.563–0.575), lo que sugiere que la calidad semántica de las respuestas generadas es equivalente con independencia del método de recuperación empleado, en el entorno controlado evaluado. Las diferencias en Fact-Coverage entre variantes en XQuAD son un artefacto de la naturaleza extractiva de los *expected facts* del benchmark y de la segmentación del corpus en chunks, no una diferencia real de calidad de respuesta. BERTScore, al capturar similitud semántica en lugar de solapamiento léxico, es la métrica más adecuada para comparar el rendimiento del sistema entre ambas evaluaciones.

En síntesis, la mayor ganancia individual proviene de la recuperación híbrida ($A \rightarrow B$: +27,5 % en cobertura factual), mientras que el reranker añade precisión al *ranking* y mejora simultáneamente el BERTScore. El orquestador contribuye con capacidades que van más allá de la cobertura factual: abstención sistemática, detección de intenciones accionables y coordinación de herramientas. La coherencia del ordenamiento de variantes en XQuAD sugiere que estos resultados no son específicos del dominio institucional, aunque su generalización a otros entornos requeriría validación adicional.

Capítulo 6

Conclusiones y trabajo futuro

Este capítulo recoge las principales aportaciones del trabajo, valora el grado de cumplimiento de los objetivos planteados en el Capítulo 3, analiza las limitaciones identificadas durante el desarrollo y la evaluación, y propone posibles líneas de ampliación y mejora futura.

6.1. Resumen de la contribución

El objetivo principal de este trabajo es diseñar, implementar y evaluar un asistente institucional privado capaz de consultar documentación interna mediante un sistema de *Retrieval-Augmented Generation* (RAG) y de apoyar acciones operativas mediante herramientas integradas. El prototipo desarrollado materializa este objetivo a través de una arquitectura modular que coordina cuatro capacidades complementarias: recuperación documental sobre un corpus institucional, mejora del *ranking* de evidencias mediante recuperación híbrida y *reranking*, orquestación del flujo de interacción mediante un agente basado en el paradigma ReAct, y generación y envío supervisado de correos electrónicos como acción operativa concreta.

Más allá del prototipo funcional, la aportación central del trabajo reside en articular una comparación controlada entre cuatro variantes del sistema, que permiten aislar el efecto de cada mejora introducida de forma progresiva. Esta comparación, apoyada en un corpus de 26 documentos y un dataset de 71 preguntas anotadas manualmente, proporciona evidencia sobre el impacto real de cada componente sobre la calidad de las respuestas generadas.

6.2. Valoración del cumplimiento de los objetivos

Los objetivos específicos planteados en el Capítulo 3 pueden valorarse del siguiente modo:

1. **Identificar requisitos funcionales y no funcionales.** Los requisitos del asistente insti-

tucional se definieron en la fase de diseño atendiendo a criterios de consulta documental, fundamentación de respuestas, trazabilidad, control del flujo y uso supervisado de herramientas. Estos requisitos orientaron todas las decisiones arquitectónicas posteriores.

2. **Diseñar una arquitectura modular basada en un agente orquestador.** Se diseñó e implementó una arquitectura que diferencia claramente entre la fase de recuperación, la fase de generación y el flujo de orquestación. El agente orquestador, basado en el paradigma ReAct, actúa como componente central de coordinación, decidiendo en cada turno qué herramientas invocar y en qué orden.
3. **Desarrollar un prototipo funcional.** El prototipo desarrollado es capaz de procesar consultas en lenguaje natural sobre documentación interna y generar respuestas fundamentadas en fragmentos recuperados del corpus. El sistema es ejecutable localmente y expone una interfaz conversacional a través de Streamlit.
4. **Integrar una funcionalidad accionable basada en herramientas.** Se implementó una herramienta de correo electrónico integrada en el flujo del asistente. El orquestador detecta la intención de envío de correo a partir de la consulta del usuario e invoca la herramienta de forma autónoma, generando un borrador que se presenta al usuario para su revisión. Solo tras la confirmación explícita del usuario, el mensaje se envía a través de la API de Microsoft Graph con autenticación OAuth 2.0; en ningún caso se produce el envío de forma automática.
5. **Establecer un *baseline* de recuperación.** La variante A, basada en búsqueda semántica densa simple con el modelo BAAI/bge-m3 y almacenamiento vectorial en Qdrant, actúa como referencia inicial. Esta configuración obtiene una cobertura de hechos esperados de 0.389 sobre el conjunto de preguntas respondibles.
6. **Implementar una variante mejorada con recuperación híbrida y *reranking*.** La variante C combina recuperación densa (Qdrant) y léxica (BM25) mediante fusión de rangos recíproca, y aplica un modelo de *reranking* (BAAI/bge-reranker-v2-m3) sobre los 20 candidatos iniciales para reducirlos a los 5 más relevantes antes de la generación. Esta variante obtiene una cobertura de hechos de 0.527, lo que representa una mejora de 13,8 puntos porcentuales respecto al *baseline*.
7. **Definir un conjunto de casos de prueba representativos.** El dataset de evaluación está formado por 71 preguntas diseñadas para cubrir distintos escenarios: preguntas respondibles desde un único documento (36), preguntas que requieren combinar información de

varias fuentes (15), preguntas comparativas (10) y preguntas no respondibles (10). Todas las preguntas respondibles cuentan con fuente esperada y hechos esperados anotados manualmente. El tamaño supera el mínimo de 25 casos establecido como criterio de comprobación.

8. **Comparar el rendimiento mediante métricas de recuperación y generación.** La evaluación incluye métricas de recuperación (Hit@1, Hit@3, MRR), métricas de calidad de respuesta (cobertura de hechos esperados, ROUGE-L F1) y métricas de similitud semántica (BERTScore F1). La comparación cubre las cuatro variantes del sistema, incluyendo el *baseline* determinista, la variante híbrida, la variante con *reranking* y el orquestador.
9. **Evaluar el impacto de la mejora del componente de recuperación.** Los resultados se valoran de forma diferenciada según el tipo de métrica:

En las **métricas de recuperación**, el criterio de mejora observable se cumple a través de Hit@3, que aumenta 6,6 puntos porcentuales (de 0.852 a 0.918), superando el umbral de 3 pp definido en la metodología. La MRR mejora en términos absolutos (de 0.712 a 0.738), pero su mejora relativa del 3,6 % no alcanza el umbral del 5 % definido inicialmente. Hit@1 desciende ligeramente (−0.017), un comportamiento habitual en sistemas híbridos con pool de candidatos ampliado. Con el tamaño de corpus y dataset de este trabajo, la diferencia no alcanza el umbral definido, aunque la dirección de la mejora es la esperada.

En las **métricas de generación**, el criterio establecido era una mejora observable en la cobertura de hechos, sin umbral numérico estricto dado el carácter más subjetivo de estas métricas. Los resultados muestran una progresión consistente desde la variante A (cobertura 0.389, BERTScore 0.643) hasta la variante C (cobertura 0.527, BERTScore 0.656), lo que sugiere que las mejoras en el componente de recuperación se trasladan de forma observable a la calidad de las respuestas generadas, en el entorno controlado evaluado.

10. **Analizar fortalezas, limitaciones y líneas de mejora.** Las principales limitaciones del sistema se recogen en la Sección 6.4, y las líneas de trabajo futuro se desarrollan en la Sección 6.5.

En conjunto, todos los objetivos específicos planteados se han cumplido satisfactoriamente dentro del alcance definido para el trabajo.

6.3. Principales resultados

El experimento de ablación muestra una tendencia de mejora progresiva al añadir cada componente del sistema. La Tabla 18 resume las cifras más relevantes.

Variante	Cobertura hechos	BERTScore F1
A: Semántico + LLM (<i>baseline</i>)	0.389	0.643
B: Híbrido + LLM	0.496	0.641
C: Híbrido + Reranker + LLM	0.527	0.656
D: Orquestador + Híbrido + Reranker	0.493	0.648

Tabla 18: Resumen de las métricas principales por variante.

La introducción de la recuperación híbrida (variante B) produce la ganancia más pronunciada en cobertura de hechos (+10,7 puntos porcentuales respecto a A), lo que sugiere que la combinación de señales densas y léxicas aporta información complementaria que la búsqueda semántica por sí sola no captura, al menos en el corpus evaluado. El *reranking* añade un incremento adicional de 3,1 puntos, que si bien es más modesto, resulta relevante porque mejora al mismo tiempo el BERTScore F1 (de 0.641 a 0.656), lo que podría indicar una mayor alineación semántica entre las respuestas y los hechos esperados. El orquestador (variante D) obtiene una cobertura de 0.493 y un BERTScore de 0.648, ligeramente por debajo de C en métricas léxicas (-0.034) pero con diferencia mínima en BERTScore (-0.008), lo que sugiere que la calidad semántica de las respuestas es equivalente. La penalización en Fact-Coverage es coherente con el estilo más elaborado del flujo ReAct, que tiende a generar respuestas estructuradas que no maximizan el solapamiento léxico con los *expected facts*. El valor diferencial del orquestador reside en su capacidad para gestionar de forma autónoma preguntas heterogéneas, detectar intenciones de acción, coordinar herramientas y mejorar la abstención (0.600 frente a 0.300–0.500 de los flujos deterministas), aspectos que las métricas de cobertura léxica no capturan directamente.

Una evaluación complementaria con GPT-5.4-mini confirma que el patrón de mejora entre variantes es robusto frente al modelo de lenguaje: la variante B alcanza FC = 0.633 y C obtiene FC = 0.549, con ganancias de hasta 0.137 puntos sobre el modelo local.

En el plano de recuperación, la variante con recuperación híbrida y *reranking* mejora Hit@3 (de 0.852 a 0.918) y MRR (de 0.712 a 0.738) respecto al *baseline*, superando los umbrales definidos en ambas métricas. Hit@1 desciende ligeramente (de 0.574 a 0.557), un comportamiento esperable en sistemas con pool de candidatos ampliado donde el reranker no siempre sitúa la fuente más relevante en primera posición. En conjunto, estos resultados sugieren que la mejora en la calidad de los fragmentos recuperados se traslada de forma observable a la

calidad de las respuestas generadas, en el entorno controlado de este trabajo.

6.4. Limitaciones

Los resultados obtenidos deben interpretarse teniendo en cuenta las siguientes limitaciones:

- **Tamaño del corpus.** El corpus de evaluación está formado por 26 documentos del dominio público de UNIR. Este tamaño es adecuado para una validación controlada del prototipo, pero insuficiente para generalizar los resultados a otros dominios institucionales o a corpus de mayor escala. El comportamiento del sistema en entornos con cientos o miles de documentos podría diferir de forma significativa.
- **Tamaño y capacidad del modelo generativo.** Los experimentos principales se realizaron con el modelo `qwen2.5:7b`, ejecutado localmente en un entorno con GPU T4. Este modelo ofrece un equilibrio razonable entre calidad y coste computacional, pero sus capacidades de razonamiento, síntesis y abstención son inferiores a las de modelos de mayor tamaño o con entrenamiento específico en dominios institucionales.
- **Dataset de evaluación.** Las 71 preguntas del dataset fueron diseñadas y anotadas manualmente por la autora del trabajo, lo que puede introducir sesgos de selección o de anotación. La ausencia de evaluadores adicionales limita la validez de las métricas de cobertura de hechos como estimación del rendimiento real del sistema en producción.
- **Ausencia de evaluación humana.** Las métricas empleadas (cobertura de hechos, ROUGE-L F1, BERTScore F1) son métricas automáticas de la calidad de las respuestas. Una evaluación humana de la utilidad, coherencia y precisión de las respuestas generadas aportaría una perspectiva complementaria que las métricas automáticas no recogen completamente.
- **Variabilidad entre ejecuciones del orquestador.** La variante D, al depender de un bucle de razonamiento iterativo con temperatura no nula, presenta cierta variabilidad entre ejecuciones. Los resultados reportados corresponden a una ejecución única sobre el conjunto completo de 61 preguntas respondibles, lo que limita la estimación de la varianza real del sistema.
- **Entorno de ejecución local.** El prototipo se ejecuta íntegramente en local, lo que simplifica la gestión de la privacidad de los datos pero limita la escalabilidad del sistema y su viabilidad en un entorno de producción institucional real.

6.5. Trabajo futuro

A partir de las limitaciones identificadas y de los resultados obtenidos, se proponen las siguientes líneas de trabajo futuro:

- **Ampliación del corpus documental.** Incorporar más fuentes institucionales, incluyendo normativas, reglamentos, guías de matrícula y documentación interna, permitiría evaluar el sistema en un contexto más próximo al de un despliegue real. Como extensión adicional, se podría replicar el sistema sobre el corpus de otra universidad de características similares para validar la generalización del enfoque.
- **Uso de modelos generativos más capaces.** Sustituir el modelo `qwen2.5:7b` por modelos de mayor tamaño o ajustados específicamente para tareas de respuesta a preguntas en español podría reducir la tasa de abstención observada y mejorar la cobertura de hechos de forma sustancial. En entornos con acceso a APIs externas, modelos de la familia Claude o GPT-4 ofrecerían una línea de comparación adicional de interés.
- **Evaluación humana.** Complementar las métricas automáticas con una evaluación por parte de usuarios reales (estudiantes o personal administrativo universitario) aportaría una medida de utilidad percibida que las métricas de solapamiento léxico o similitud semántica no capturan.
- **Ampliación del dataset de evaluación.** Incrementar el número de preguntas del dataset, especialmente en las categorías de preguntas comparativas y preguntas que requieren combinar múltiples fuentes, mejoraría la robustez estadística de los resultados y reduciría el efecto de la varianza por tamaño de muestra.
- **Mejora del componente de herramientas.** El prototipo actual incluye una herramienta de correo electrónico. En un sistema más completo, el catálogo de herramientas podría ampliarse para incluir la consulta de calendarios académicos, la generación de instancias administrativas o la integración con sistemas de gestión universitaria, incrementando la utilidad operativa del asistente.
- **Capacidad de clarificación interactiva.** El orquestador actual no puede solicitar información adicional al usuario cuando la consulta es ambigua o incompleta. Incorporar una herramienta de clarificación permitiría al agente formular preguntas dirigidas, tanto al inicio de la conversación como durante el proceso de razonamiento, mejorando la calidad de las respuestas en escenarios donde la intención del usuario no queda suficientemente especificada en el primer turno.

- **Despliegue en producción.** El paso de un prototipo local a un sistema desplegado en producción implicaría abordar aspectos de seguridad, autenticación de usuarios, control de acceso a documentos sensibles, escalabilidad del componente de recuperación y supervisión del comportamiento del sistema en producción.

6.6. Reflexión final

Este trabajo ha demostrado que es posible construir un asistente institucional privado, modular y evaluable combinando tecnologías de recuperación híbrida, *reranking* y orquestación basada en modelos de lenguaje locales, sin depender de servicios externos de pago ni comprometer la privacidad de los datos. Los resultados muestran una tendencia de mejora progresiva al añadir cada componente al *pipeline*, con la recuperación híbrida como la aportación de mayor impacto individual, y el orquestador aportando valor en dimensiones cualitativas que las métricas léxicas no capturan completamente.

Al mismo tiempo, la evaluación evidencia que las limitaciones más relevantes no residen únicamente en la arquitectura del sistema, sino en factores externos como el tamaño del modelo generativo, la representatividad del corpus y la dificultad inherente de evaluar calidad conversacional mediante métricas automáticas. Estas limitaciones no invalidan los resultados, sino que los contextualizan como una contribución inicial y bien delimitada, sobre la que futuras iteraciones del trabajo podrían construir con mayor alcance y recursos.

En definitiva, el prototipo desarrollado constituye una base sólida para explorar cómo los sistemas RAG con orquestación pueden aplicarse de forma útil y controlada en entornos institucionales, abriendo la puerta a asistentes que no solo informan, sino que acompañan al usuario en el siguiente paso de su gestión.

Bibliografía

- Cormack, Gordon V., Charles L. A. Clarke y Stefan Buettcher (2009). «Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods». En: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, págs. 758-759.
- Liu, Tie-Yan (2009). *Learning to Rank for Information Retrieval*. Springer. doi: [10.1007/978-3-642-14267-3](https://doi.org/10.1007/978-3-642-14267-3).
- Robertson, Stephen y Hugo Zaragoza (2009). «The Probabilistic Relevance Framework: BM25 and Beyond». En: *Foundations and Trends in Information Retrieval* 3.4, págs. 333-389. doi: [10.1561/15000000019](https://doi.org/10.1561/15000000019).
- Meyer, Carl (2011). *PEP 405 – Python Virtual Environments*. Accessed: 2026-05-19. url: <https://peps.python.org/pep-0405/>.
- Pedregosa, Fabian y col. (2011). «Scikit-learn: Machine Learning in Python». En: *Journal of Machine Learning Research* 12.85, págs. 2825-2830. url: <https://www.jmlr.org/papers/v12/pedregosa11a.html>.
- Artetxe, Mikel, Sebastian Ruder y Dani Yogatama (2020). «On the Cross-lingual Transferability of Monolingual Representations». En: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, págs. 4623-4637. doi: [10.18653/v1/2020.acl-main.421](https://doi.org/10.18653/v1/2020.acl-main.421).
- Harris, Charles R. y col. (2020). «Array programming with NumPy». En: *Nature* 585, págs. 357-362. doi: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- Karpukhin, Vladimir y col. (2020). «Dense Passage Retrieval for Open-Domain Question Answering». En: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Online: Association for Computational Linguistics, págs. 6769-6781. doi: [10.18653/v1/2020.emnlp-main.550](https://doi.org/10.18653/v1/2020.emnlp-main.550).
- Lewis, Patrick y col. (2020). «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks». En: *Advances in Neural Information Processing Systems*.

- Nogueira, Rodrigo y col. (2020). «Document Ranking with a Pretrained Sequence-to-Sequence Model». En: *Findings of the Association for Computational Linguistics: EMNLP 2020*. Online: Association for Computational Linguistics, págs. 708-718. doi: [10.18653/v1/2020.findings-emnlp.63](https://doi.org/10.18653/v1/2020.findings-emnlp.63). url: <https://aclanthology.org/2020.findings-emnlp.63/>.
- Virtanen, Pauli y col. (2020). «SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python». En: *Nature Methods* 17.3, págs. 261-272. doi: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- Wolf, Thomas y col. (2020). «Transformers: State-of-the-Art Natural Language Processing». En: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, págs. 38-45. doi: [10.18653/v1/2020.emnlp-demos.6](https://doi.org/10.18653/v1/2020.emnlp-demos.6). url: <https://aclanthology.org/2020.emnlp-demos.6/>.
- Qu, Yingqi y col. (2021). «RocketQA: An Optimized Training Approach to Dense Passage Retrieval for Open-Domain Question Answering». En: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Online: Association for Computational Linguistics, págs. 5835-5847. doi: [10.18653/v1/2021.naacl-main.466](https://doi.org/10.18653/v1/2021.naacl-main.466).
- Kwon, Woosuk y col. (2023). «Efficient Memory Management for Large Language Model Serving with PagedAttention». En: *Proceedings of the 29th Symposium on Operating Systems Principles*, págs. 611-626. doi: [10.1145/3600006.3613165](https://doi.org/10.1145/3600006.3613165).
- Li, Minghao y col. (2023). «API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs». En: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, págs. 3102-3116. doi: [10.18653/v1/2023.emnlp-main.187](https://doi.org/10.18653/v1/2023.emnlp-main.187).
- Ma, Xinbei y col. (2023). «Query Rewriting for Retrieval-Augmented Large Language Models». En: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, págs. 5303-5315. doi: [10.18653/v1/2023.emnlp-main.322](https://doi.org/10.18653/v1/2023.emnlp-main.322).
- Schick, Timo y col. (2023). «Toolformer: Language Models Can Teach Themselves to Use Tools». En: *Advances in Neural Information Processing Systems*.
- Yao, Shunyu y col. (2023). «ReAct: Synergizing Reasoning and Acting in Language Models». En: *The Eleventh International Conference on Learning Representations*.
- Asai, Akari y col. (2024). «Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection». En: *The Twelfth International Conference on Learning Representations*.
- Chen, Jianlv y col. (2024a). «BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation». En: *Findings of the Association for*

- Computational Linguistics: ACL 2024*. Association for Computational Linguistics, págs. 8753-8768. url: <https://aclanthology.org/2024.findings-acl.137/>.
- Chen, Weize y col. (2024b). «AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors». En: *The Twelfth International Conference on Learning Representations*.
- Hong, Sirui y col. (2024). «MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework». En: *The Twelfth International Conference on Learning Representations*.
- Jeong, Soyeong y col. (2024). «Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity». En: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, págs. 7036-7050. doi: [10.18653/v1/2024.naacl-long.389](https://doi.org/10.18653/v1/2024.naacl-long.389).
- Patil, Shishir G. y col. (2024). «Gorilla: Large Language Model Connected with Massive APIs». En: *Advances in Neural Information Processing Systems*. doi: [10.52202/079017-4020](https://doi.org/10.52202/079017-4020).
- Qin, Yujia y col. (2024). «ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs». En: *The Twelfth International Conference on Learning Representations*.
- Saad-Falcon, Jon y col. (2024). «ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems». En: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Mexico City, Mexico: Association for Computational Linguistics, págs. 338-354. doi: [10.18653/v1/2024.naacl-long.20](https://doi.org/10.18653/v1/2024.naacl-long.20).
- Shi, Weijia y col. (2024). «REPLUG: Retrieval-Augmented Black-Box Language Models». En: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, págs. 8371-8384. doi: [10.18653/v1/2024.naacl-long.463](https://doi.org/10.18653/v1/2024.naacl-long.463).
- Wang, Liang y col. (2024). «Multilingual E5 Text Embeddings: A Technical Report». En: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, págs. 11089-11102. doi: [10.18653/v1/2024.emnlp-main.618](https://doi.org/10.18653/v1/2024.emnlp-main.618).
- Wu, Qingyun y col. (2024). «AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations». En: *Proceedings of the First Conference on Language Modeling*.
- Zeng, Shenglai y col. (2024). «The Good and The Bad: Exploring Privacy Issues in Retrieval-Augmented Generation (RAG)». En: *Findings of the Association for Computational Linguistics: ACL 2024*, págs. 4505-4524. doi: [10.18653/v1/2024.findings-acl.267](https://doi.org/10.18653/v1/2024.findings-acl.267).

- Dang, Yufan y col. (2025). «Multi-Agent Collaboration via Evolving Orchestration». En: *Advances in Neural Information Processing Systems*.
- Kalra, Rishi y col. (abr. de 2025). «HyPA-RAG: A Hybrid Parameter Adaptive Retrieval-Augmented Generation System for AI Legal and Policy Applications». En: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track)*. Albuquerque, New Mexico: Association for Computational Linguistics, págs. 1036-1054. doi: [10.18653/v1/2025.naacl-industry.79](https://doi.org/10.18653/v1/2025.naacl-industry.79).
- Liu, Hanchao y col. (abr. de 2025). «WorkTeam: Constructing Workflows from Natural Language with Multi-Agents». En: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track)*. Albuquerque, New Mexico: Association for Computational Linguistics, págs. 20-35. doi: [10.18653/v1/2025.naacl-industry.3](https://doi.org/10.18653/v1/2025.naacl-industry.3).
- Nguyen, Long S. T. y col. (dic. de 2025). «When in Doubt, Ask First: A Unified Retrieval Agent-Based System for Ambiguous and Unanswerable Question Answering». En: *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*. Mumbai, India: The Asian Federation of Natural Language Processing y The Association for Computational Linguistics, págs. 452-472. doi: [10.18653/v1/2025.findings-ijcnlp.27](https://doi.org/10.18653/v1/2025.findings-ijcnlp.27).
- Qwen Team (2025). *Qwen2.5 Technical Report*. arXiv: [2412.15115](https://arxiv.org/abs/2412.15115) [cs.CL].
- Trienes, Jan y col. (2025). «Marcel: A Lightweight and Open-Source Conversational Agent for University Student Support». En: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Suzhou, China: Association for Computational Linguistics, págs. 181-195. url: <https://aclanthology.org/2025.emnlp-demos.13/>.
- Zhang, Xuan y col. (nov. de 2025). «AskToAct: Enhancing LLMs Tool Use via Self-Correcting Clarification». En: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Suzhou, China: Association for Computational Linguistics, págs. 13484-13511. doi: [10.18653/v1/2025.emnlp-main.682](https://doi.org/10.18653/v1/2025.emnlp-main.682).
- Anand, Matthew (2026). *python-markdownify: Convert HTML to Markdown*. Accessed: 2026-05-19. url: <https://github.com/matthewwithanm/python-markdownify>.
- Arize AI (2026). *Phoenix Documentation*. Accessed: 2026-05-19. url: <https://arize.com/docs/phoenix>.

- Chroma (2026). *Chroma Documentation*. <https://docs.trychroma.com/>. Accessed: 2026-05-19.
- Docker Inc. (2026a). *Docker Compose documentation*. Accessed: 2026-05-19. url: <https://docs.docker.com/compose/>.
- (2026b). *Persisting container data*. Accessed: 2026-05-19. url: <https://docs.docker.com/get-started/docker-concepts/running-containers/persisting-container-data/>.
- Meta AI (2026). *FAISS Documentation*. <https://faiss.ai/>. Accessed: 2026-05-19.
- Ollama (2026). *OpenAI compatibility*. Accessed: 2026-05-19. url: <https://docs.ollama.com/api/openai-compatibility>.
- OpenTelemetry (2026). *OpenTelemetry Documentation*. Accessed: 2026-05-19. url: <https://opentelemetry.io/docs/>.
- Pinecone (2026). *Pinecone Documentation*. <https://docs.pinecone.io/>. Accessed: 2026-05-19.
- Qdrant (2026). *Qdrant Documentation*. <https://qdrant.tech/documentation/>. Accessed: 2026-05-19.
- Sentence Transformers (2026). *Cross-Encoders*. Accessed: 2026-05-19. url: https://www.sbert.net/examples/cross_encoder/applications/README.html.
- Streamlit (2026). *Streamlit Documentation*. Accessed: 2026-05-19. url: <https://docs.streamlit.io/>.
- Weaviate (2026). *Weaviate Documentation*. <https://weaviate.io/developers/weaviate>. Accessed: 2026-05-19.

Apéndices

Apéndice A

Recursos del proyecto

Este apéndice recoge los recursos digitales asociados al trabajo: el repositorio de código fuente y la grabación de la demostración funcional del sistema.

Código fuente

El código fuente completo del prototipo —incluyendo el pipeline RAG, el orquestador, las herramientas integradas, los scripts de evaluación y la interfaz Streamlit— está disponible en GitHub, junto con instrucciones de instalación y puesta en marcha en el archivo `README.md`:

<https://github.com/nuriaiglesias/private-assistant-rag>

Demostración del sistema

Se ha grabado una demostración del sistema en funcionamiento que ilustra las principales capacidades del asistente: consulta sobre documentación institucional, abstención ante preguntas no respondibles y activación supervisada de la herramienta de correo electrónico:

https://drive.google.com/file/d/16hjMI20DtRpueGSbksihCKch5a_2R5Pl/view?usp=sharing